



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

THỰC HÀNH PHƯƠNG PHÁP TÍNH

KHOA ĐIỆN TỬ VIỄN THÔNG

Năm 2025

Bài 1: CÁC LỆNH CƠ BẢN TRONG MATLAB VÀ ỨNG DỤNG TRONG PHÂN GIẢI MẠCH

A. LÝ THUYẾT

1. Những lệnh cơ bản sử dụng trong command window của Matlab:

Lệnh/dấu	Ý nghĩa
exit hoặc quit	Thoát khỏi MATLAB
help	Tra cứu thông tin trong MATLAB:
help xyz	Tìm hiểu thông tin của 1 hàm cụ thể xyz
lookfor abc	Tìm kiếm lệnh có chứa một chuỗi kí tự abc nào đó
clear	Xoá hiện có trong workspace
clc	Xoá cửa sổ command window
;	Kết thúc 1 lệnh không hiện thị kết quả
%	Viết comment. Những gì sau dấu % đều không được MATLAB thực hiện
edit	Mở một bản soạn thảo code, saved lại thành file đuôi .m
disp	Hiển thị giá trị các biến

2. Những phép toán cơ bản (đại số, vector, ma trận, số phức):

Lệnh/dấu	Ý nghĩa
[...]	Khởi tạo ma trận
, hoặc dấu cách	Bắt đầu tạo mới 1 phần tử trên 1 hàng
;	Tạo mới 1 hàng trong ma trận
length(a)	Trả về chiều dài của ma trận a
+ - *	Cộng/trừ/nhân đại số/vector/ma trận
/	Phép chia đại số/Chia phải ma trận
\	Chia trái ma trận
^	Luỹ thừa
.+ .- .* ./ .\ .^	Thực hiện phép toán trên từng phần tử của vector/ma trận
log(a)	ln của a
logb(c)	$\log_b(c)$
sqrt(a)	Khai căn bậc 2 của a
factorial(a)	Giai thừa của a
zeros(a,b)	Tạo ma trận 0, a hàng, b cột
one(a,b)	Tạo ma trận 1, a hàng, b cột
:	Toàn bộ các phần tử thuộc cột hoặc dòng trong ma trận
a:b	Tạo một vector [a, a+1, a+2, ... b]
a:c:b	Tạo một vector [a, a+c, a+2c, ... b]
'	Chuyển vị
fliplr(a)	Đảo ma trận a từ trái qua phải
flipud(a)	Đảo ma trận a từ trên xuống dưới

round(a)	Làm tròn a tới số nguyên gần nhất
fix(a)	Làm tròn hướng về 0
floor(a)	Làm tròn xuống
ceil(a)	Làm tròn lên
factor(a)	Tìm các thừa số nguyên tố của a
max(a)/min(a)	Phần tử lớn nhất/nhỏ nhất trong ma trận a
size(a)	Cho biết kích thước của ma trận
rand()	Tạo một ma trận có các phần tử random từ 0->1
i/j	Phần tử phức
complex(a,b)	Tạo một số phức $a+ib$
real(a)	Phần thực của số phức a
imag(a)	Phần ảo của số phức a
abs(a)	Giá trị tuyệt đối của số phức a
angle(a)	Giá trị pha của số phức a
pi	π
~	Phủ định
~=	Không bằng
>=	Lớn hơn hoặc bằng
<=	Nhỏ hơn hoặc bằng
<	Nhỏ hơn
>	Lớn hơn
==	Bằng (trong phép toán logic)

3. Những lệnh cơ bản để vẽ đồ thị, biểu đồ:

Lệnh/dấu	Ý nghĩa
plot()	Vẽ đồ thị 2 trục tọa độ
ezplot	Vẽ 1 hàm xác định
grid on/off	Bật tắt lưới trong đồ thị
title('')	Tên của đồ thị
xlabel('')	Tên của trục x
ylabel('')	Tên của trục y
hold on/off	Vẽ nhiều đồ thị trên cùng một hình
figure(a)	Tạo một cửa sổ tên a để vẽ hình
axis([a, b, c, d])	Tạo trục x từ a ->b, trục y từ c ->d
text(a, b, 'text')	Thêm chữ tại tọa độ (a, b)
legend('a', 'b',...)	Thêm ghi chú cho các đồ thị

4. Những lệnh cấu trúc rẽ nhánh và lặp cơ bản:

If	For	While
if (biểu thức logic) Lệnh #1; Lệnh #2; ... else Lệnh #1; Lệnh #2;	for (chỉ số:chiều dài của mảng) Lệnh #1; Lệnh #2; Lệnh #3; ... end	while (biểu thức logic) Lệnh #1; Lệnh #2; Lệnh #3; ... end

...		
end		

B. THỰC HÀNH

Bài 1

Trong định luật Ohm, hiệu điện thế giữa 2 đầu điện trở $v = Ri$, trong đó i là dòng điện, R là điện trở. Công suất tiêu tán của điện trở R là: $P = Ri^2$. Giả sử $R = 10\Omega$. Cho i từ 0 đến 10A bước 1A.

- Tìm v và P tương ứng với dòng điện trên
- Vẽ v và P theo dòng điện i

Bài 2

Cho công thức điện thế khi nạp của một tụ điện như sau:

$$v(t) = 10(1 - e^{-0.2t})$$

Cho t từ 0 đến 50 giây, bước 1s, vẽ dạng sóng của $v(t)$

Gợi ý: Sử dụng hàm $\exp(x)$ để tính giá trị e^x

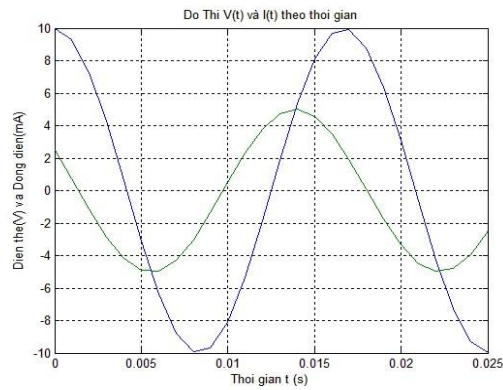
Bài 3

Trong mạch R-L, điện thế và dòng điện được biểu diễn theo thời gian như sau::

$$v(t) = 10\cos(377t)$$

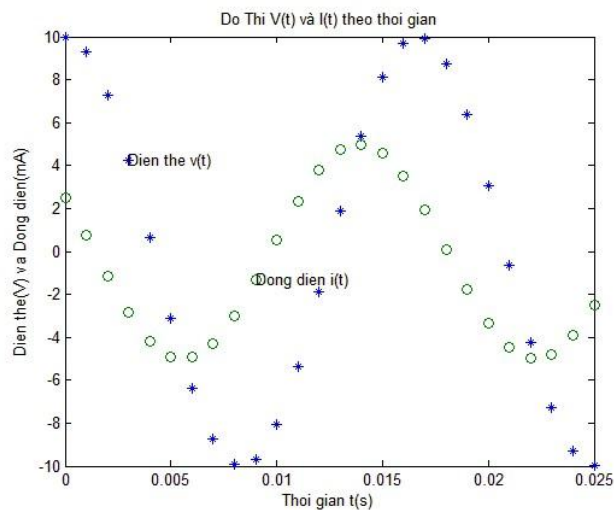
$$i(t) = 5\cos(377t + 60^\circ)$$

- Viết m-file (file script) để vẽ 2 đồ thị của hàm $v(t)$ và $i(t)$ theo thời gian (cho $t = 0:0.001:0.025$) trên cùng một hình như hình 1.



Hình 1: Đồ thị bài 3a

- Viết m-file để vẽ 2 đồ thị của hàm $v(t)$ và $i(t)$ theo thời gian (cho $t = 0:0.001:0.025$) trên cùng 1 hình như hình 2.
- Viết m-file xuất ra 1 cửa sổ có cả 2 hình vừa vẽ với lệnh subplot.
- Viết m-file xuất ra 2 cửa sổ có 2 hình vừa vẽ với lệnh figure.



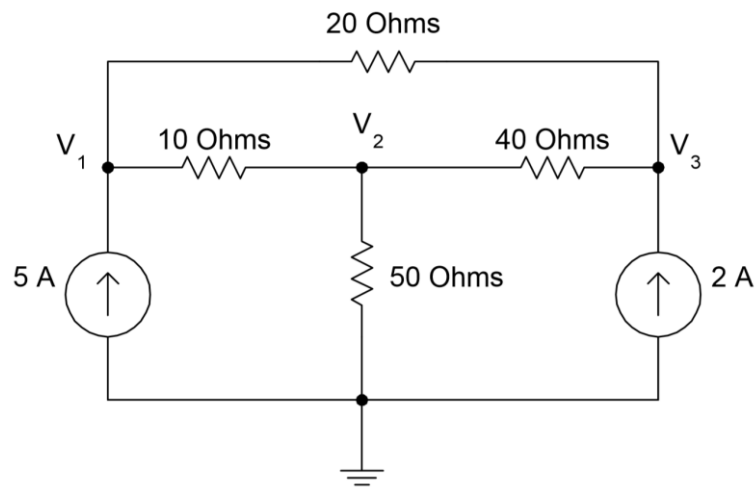
Hình 2: Đồ thị bài 3b.

Gợi ý: Tìm hiểu thêm về các lệnh plot, subplot, title, xlabel, ylabel, figure trong Matlab help.

Sử dụng hàm cosd để tính giá trị cosin ở độ, hàm text() để ghi chú lên hình

Bài 4

Viết m-file tính điện thế V_1, V_2, V_3 của mạch điện ở hình 3 (Gợi ý: Tính tay các phương trình rồi viết m-file chia ma trận).



Hình 3: Sơ đồ mạch bài 4.

Hướng dẫn:

Chọn chiều dòng điện dương hướng ra nút, dựa vào định luật Kirchhoff về dòng điện ta có:

(Áp dụng tổng dẫn riêng và tổng dẫn hỗ tương của nút)

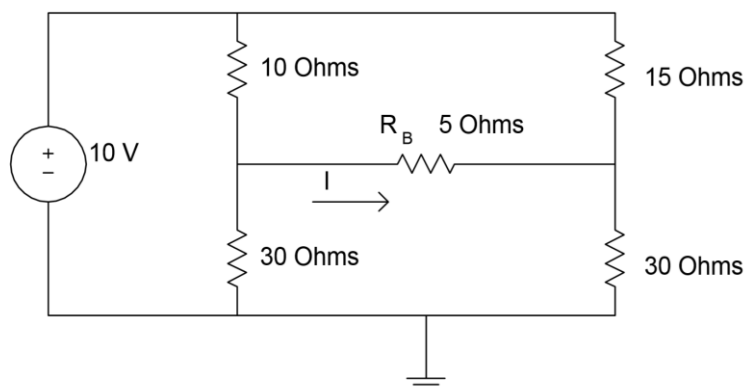
$$\begin{cases} \left(\frac{1}{10} + \frac{1}{20} \right) V_1 - \frac{1}{10} V_2 - \frac{1}{20} V_3 - 5 = 0 \\ -\frac{1}{10} V_1 + \left(\frac{1}{10} + \frac{1}{50} + \frac{1}{40} \right) V_2 - \frac{1}{40} V_3 = 0 \\ -\frac{1}{20} V_1 - \frac{1}{40} V_2 + \left(\frac{1}{20} + \frac{1}{40} \right) V_3 - 2 = 0 \end{cases}$$

Viết dưới dạng ma trận $A.V = I$

$$v = 404.2857; 350.0000; 412.8571$$

Bài 5

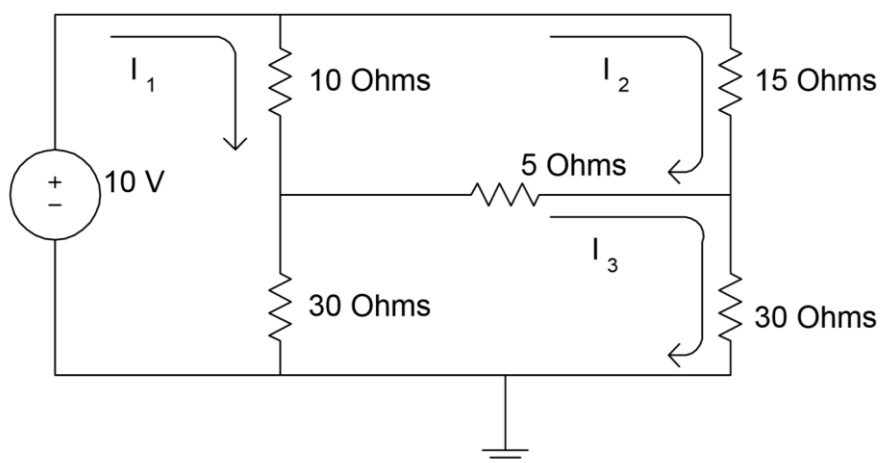
Tìm dòng điện I qua R_B trong mạch điện sau



Hướng dẫn:

Áp dụng định luật Kirchhoff về lưới.

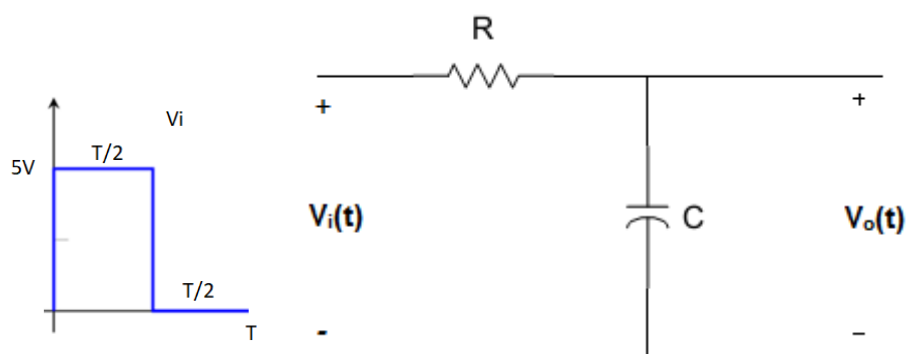
Chọn chiều dòng điện như hình vẽ



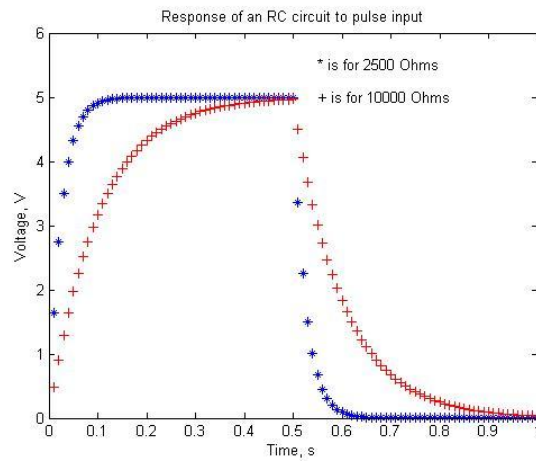
$$\begin{cases} (10 + 30)I_1 & -10I_2 & -30I_3 & -10 & = 0 \\ -10I_1 & +(10 + 15 + 5)I_2 & -5I_3 & & = 0 \\ -30I_1 & -5I_2 & +(5 + 30 + 30)I_3 & & = 0 \end{cases}$$

Bài 6

Cho mạch điện RC như hình vẽ, ngõ vào $V_i(t)$ là một xung vuông biên độ là 5V, độ rộng xung $T/2 = 0.5s$. $C = 10\mu F$. Giả sử điện thế ban đầu của tụ bằng 0.



Viết một m-file vẽ 2 đồ thị $V_o(t)$ theo t ứng với $R = 2.5k\Omega$ và $R = 10k\Omega$ trên cùng 1 hình.



Hướng dẫn:

Nửa chu kỳ đầu (0 đến $T/2$), $V_i = V_s = 5V$, tụ nạp với phương trình

$$V_o(t) = V_s \left(1 - e^{-\frac{t}{RC}} \right),$$

Nửa chu kỳ sau ($T/2$ đến T), tụ xả với phương trình

$$V_o(t) = V_m \cdot e^{-\frac{t}{RC}},$$

V_m là giá trị điện thế khi tụ nạp ở thời gian $T/2$

PHỤ LỤC

NGÔN NGỮ MATLAB VÀ C

I. Kiểu dữ liệu cơ bản

1. Toán tử 2 ngôi

Toán tử	Độ ưu tiên (0 là cao nhất)	MATLAB	C
$a \wedge b$	0	Hàm mũ, a mũ b	Toán tử XOR trên bit
$a * b$	1	Thực hiện phép nhân	Thực hiện phép nhân
a / b	1	Thực hiện phép chia	Thực hiện phép chia
$a \% b$	1	Không hỗ trợ. Sử dụng hàm rem hoặc mod để thay thế	Phép chia lấy dư
$a + b$	2	Thực hiện phép cộng	Thực hiện phép cộng
$a - b$	2	Thực hiện phép trừ	Thực hiện phép trừ
$a \gg b$	3	Không hỗ trợ. Sử dụng hàm bitshift	Dịch a sang phải b bits.
$a \ll b$	3	Không hỗ trợ. Sử dụng hàm bitshift	Dịch a sang trái b bits.
$a > b$	4	So sánh lớn hơn.	So sánh lớn hơn.
$a < b$	4	So sánh nhỏ hơn.	So sánh nhỏ hơn.
$a \geq b$	4	So sánh lớn hơn hoặc bằng.	So sánh lớn hơn hoặc bằng.
$a \leq b$	4	So sánh nhỏ hơn hoặc bằng	So sánh nhỏ hơn hoặc bằng
$a == b$	5	So sánh bằng	So sánh bằng
$a \sim b$	5	So sánh không bằng	So sánh không bằng
$a \neq b$	5	Không hỗ trợ. Sử dụng $a \sim b$.	So sánh không bằng
$a <> b$	5	Không hỗ trợ. Sử dụng $a \sim b$.	So sánh không bằng
$a \& b$	6	Toán tử logic AND. Sử dụng hàm bitand để thực hiện toán tử AND trên bit	Bitwise AND
$a \wedge b$	7	Không hỗ trợ. Sử dụng hàm bitxor để thực hiện toán tử XOR trên bit	Bitwise XOR

a b	8	Toán tử logic OR. Sử dụng hàm bitor để thực hiện toán tử OR trên bit	Bitwise OR
a && b	9	Toán tử logic AND.	Logical AND.
a b	10	Toán tử logic OR.	Logical OR.

2. Toán tử 1 ngôi

Toán tử	MATLAB	C
~a	Toán tử Logic NOT. Đối tử toán tử NOT trên bit (bitwise) sử dụng hàm bitcmp	Toán tử bitwise NOT
!a	Không hỗ trợ, sử dụng ~a để thay thế.	Toán tử logic NOT
-a	Số âm	Số âm
a++	Không hỗ trợ	Tăng một đơn vị. Tương đương $a = a + 1$.
a--	Không hỗ trợ	Giảm một đơn vị. Tương đương $a = a - 1$.

3. Toán tử gán

Toán tử	MATLAB	C
a = b	Phép gán thông thường	Phép gán thông thường
a += b	Không hỗ trợ, sử dụng $a = a + b$ để thay thế.	Tương đương $a = a + b$.
a -= b	Không hỗ trợ, sử dụng $a = a - b$ để thay thế.	Tương đương $a = a - b$.
a *= b	Không hỗ trợ, sử dụng $a = a * b$ để thay thế.	Tương đương $a = a * b$.
a /= b	Không hỗ trợ, sử dụng $a = a / b$ để thay thế.	Tương đương $a = a / b$.
a &= b	Không hỗ trợ, sử dụng $a = \text{bitand}(a, b)$ để thay thế.	Tương đương $a = a \& b$ (bitwise AND).
a ^= b	Không hỗ trợ, sử dụng $a = \text{bitxor}(a, b)$ để thay thế.	Tương đương $a = a \wedge b$ (bitwise XOR)
a = b	Không hỗ trợ, sử dụng $a = \text{bitor}(a, b)$ để thay thế.	Tương đương $a = a b$ (bitwise OR)

II. Vector và Ma trận

1. Chỉ số và cách truy xuất

Nội dung	Ngôn ngữ Matlab	Ngôn ngữ C
Phần tử đầu tiên của vector V	V(1)	V[0]
Phần tử thứ I của vector V	V(i)	V[i-1]
Phần tử hàng 4 cột 5 của ma trận M	M(4,5)	M[3][4]
Phần tử hàng i cột j của ma trận M	M(i,j)	M[i-1][j-1]

2. Toán tử 2 ngôi

Toán tử	Độ ưu tiên	MATLAB	C
a * b	1	Nhân 2 ma trận	Áp dụng cho nhân từng phần tử
a .* b	1	Nhân từng phần tử với nhau Element-wise multiplication.	Không hỗ trợ
a / b	1	Chia phải ma trận	Áp dụng cho chia từng phần tử
a ./ b	1	Chia phải từng phần tử của ma trận	Không hỗ trợ
a \ b	1	Chia trái ma trận	Không hỗ trợ
a .\ b	1	Chia trái từng phần tử của ma trận	Không hỗ trợ
a + b	2	Thực hiện cộng	Thực hiện cộng
a - b	2	Thực hiện trừ	Thực hiện trừ
a == b	3	So sánh bằng	So sánh bằng
a ~= b	3	So sánh không bằng	So sánh không bằng
a != b	3	Không hỗ trợ, thay thế bằng a ~= b.	Không hỗ trợ, thay thế bằng a ~= b.
a <> b	3	Không hỗ trợ, thay thế bằng a ~= b.	Không hỗ trợ, thay thế bằng a ~= b.

3. Toán tử 1 ngôi

Toán tử	MATLAB	C
---------	--------	---

~a	Toán tử Logic NOT. Đối tử toán tử NOT trên bit (bitwise) sử dụng hàm bitcmp	Toán tử bitwise NOT
!a	Không hỗ trợ, sử dụng ~a để thay thế.	Toán tử logic NOT
-a	Số âm	Số âm
a++	Không hỗ trợ	Tăng tất cả các phần tử của vector hay ma trận. Tương đương $a = a + 1$.
a--	Không hỗ trợ	Giảm tất cả các phần tử của vector hay ma trận. Tương đương $a = a - 1$.

4. Toán tử gán

Toán tử	MATLAB	C
a = b	Phép gán thông thường	Phép gán thông thường
a += b	Không hỗ trợ, sử dụng $a = a + b$ để thay thế.	Tương đương $a = a + b$.
a -= b	Không hỗ trợ, sử dụng $a = a - b$ để thay thế.	Tương đương $a = a - b$.
a *= b	Không hỗ trợ, sử dụng $a = a * b$ để thay thế.	Tương đương $a = a * b$.
a /= b	Không hỗ trợ, sử dụng $a = a / b$ để thay thế.	Tương đương $a = a / b$.
a &= b	Không hỗ trợ, sử dụng $a = \text{bitand}(a, b)$ để thay thế.	Tương đương $a = a \& b$ (bitwise AND).
a ^= b	Không hỗ trợ, sử dụng $a = \text{bitxor}(a, b)$ để thay thế.	Tương đương $a = a \wedge b$ (bitwise XOR)
a = b	Không hỗ trợ, sử dụng $a = \text{bitor}(a, b)$ để thay thế.	Tương đương $a = a b$ (bitwise OR)

Ví dụ	MATLAB	C
Khai báo ma trận A có giá trị là $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$	$A = [1 \ 2 \ 3; 4 \ 5 \ 6];$	$\text{int } A[2][3] = \{\{1, 2, 3\}, \{4, 5, 6\}\};$
Gán giá trị 10 vào phần tử đầu tiên của vector V.	$V(1) = 10;$	$V[0] = 10;$
Gán giá trị 77 vào phần tử hàng 2 cột 9 của ma trận M.	$M(2, 9) = 77;$	$M[1][8] = 77;$

III. Cú pháp lặp và điều kiện

Cú pháp	MATLAB	C
---------	--------	---

If else	<pre> if expression statements elseif expression statements else statements end Ví dụ x = 10; minVal = 2; maxVal = 6; if (x >= minVal) && (x <= maxVal) disp('Value within specified range.') elseif (x > maxVal) disp('Value exceeds maximum value.') else disp('Value is below minimum value.') end </pre>	<pre> if (test expression1) { // statement(1) } else if(test expression2) { // statement(2) } else if (test expression3) { // statement(3) } . . else { // statement(n) } </pre>
Switch case	<pre> switch switch_expression case case_expression statements case case_expression statements ... otherwise statements end Ví dụ: n = input('Enter a number: '); switch n case -1 disp('negative one') </pre>	<pre> switch (x) { case value1: // If x is equal to value1, the code in this block will be executed break; case value2: // If x is equal to value2, the code in this block will be executed break; case value3: // If x is equal to value3, the code in this block will be executed break; ... case valuen: // If x is equal to valuen, the code in this block will be executed break; </pre>

	<pre> case 0 disp('zero') case 1 disp('positive one') otherwise disp('other value') end </pre>	<pre> default: /* If x is not equal to one of the values above the code in this block will be executed */ break; } </pre>
For	<pre> for index = values statements end Ví dụ 1: for v = 1.0:-0.2:0.0 disp(v) end Ví dụ 2: for v = [1 5 8 17] disp(v) end </pre>	<pre> for (init; condition; increment) { statement(s); } Ví dụ: for (int i = 1; i <= 5; i++) { printf("%d ", i); } </pre>
While	<pre> while expression statements end Ví dụ 1: n = 10; f = n; while n > 1 n = n-1; f = f*n; end disp(['n! = ' num2str(f)]) Ví dụ 2: limit = 0.8; s = 0; while 1 tmp = rand; if tmp > limit break </pre>	<pre> while(condition) { statement(s); } Ví dụ: while (n != 0) { printf("%d ", n); n--; } </pre>

	<pre> end s = s + tmp; end </pre>	
continue	<pre> for n = 1:50 if mod(n,7) continue end disp(['Divisible by 7: ' num2str(n)]) end </pre>	continue
break	<pre> limit = 0.8; s = 0; while 1 tmp = rand; if tmp > limit break end s = s + tmp; end </pre>	break

IV. Input và Output

Hàm	MATLAB	C
disp	Dùng hàm disp để hiển thị kết quả của biến <pre> name = 'Alice'; age = 12; X = [name, ' will be ', num2str(age), ' this year.']; disp(X) </pre>	
sprintf	Dùng hàm sprintf để tạo 1 chuỗi <pre> name = 'Alice'; age = 12; X = sprintf('%s will be %d this year.', name, age); disp(X) sprintf với định dạng A = pi*ones(1,3); txt = sprintf('%f %.2f %12f', A) </pre>	
fprintf	Dùng hàm fprintf để hiển thị với định dạng <pre> name = 'Alice'; age = 12; </pre>	<pre> int n=30; </pre>

	<code>fprintf('%s will be %d this year.\n',name,age);</code>	<code>printf("n = %d",n); //sẽ in ra: n = 30</code>
input	Dùng hàm input để nhập vào từ bàn phím <pre>prompt = 'What is the original value?'; x = input(prompt) y = x*10</pre> Đối với phiên bản Matlab mới, kiểu dữ liệu input có thể là số, chuỗi hoặc hàm Nếu chỉ input dạng text ta sử dụng cú pháp <pre>str = input(prompt, 's')</pre>	<pre>int scanf(const char *format, ...)</pre> Ví dụ: <pre>char str1[20]; printf("Nhap ten: "); scanf("%s", &str1);</pre>

Một số định dạng khi dùng hàm `fprintf`

Kiểu dữ liệu	Mã	Chi tiết
Số nguyên có dấu	<code>%d</code> or <code>%i</code>	Hệ 10
Số nguyên không dấu	<code>%u</code>	Hệ 10
	<code>%o</code>	Hệ 8 (octal)
	<code>%x</code>	Hệ 16 (hexadecimal), sử dụng ký tự thường a–f
	<code>%X</code>	Giống <code>%x</code> , sử dụng ký tự in A–F
Số thực	<code>%f</code>	Định dạng số thực. Ví dụ <code>%.2f</code> định dạng 2 chữ số thập phân sau dấu phẩy
	<code>%e</code>	Định dạng với số mũ, ví dụ 3.141593e+00
	<code>%E</code>	Giống <code>%e</code> , nhưng sử dụng E in, ví dụ 3.141593E+00
	<code>%g</code>	Định dạng rút gọn của <code>%e</code> hoặc <code>%f</code> , các số 0 sau cùng sẽ được loại bỏ
	<code>%G</code>	Định dạng rút gọn của <code>%E</code> hoặc <code>%f</code> , các số 0 sau cùng sẽ được loại bỏ
Ký tự hoặc chuỗi	<code>%c</code>	Ký tự đơn
	<code>%s</code>	Vector ký tự hoặc mảng chuỗi.

Bài 2: HÀM TRONG MATLAB

C. LÝ THUYẾT

1. Cú pháp

```
function [y1,...,yN] = myfun(x1,...,xM)
```

Trong đó:

function: từ khóa để khai báo hàm

[y1,...,yN]: Khai báo các output của hàm

myfun: Tên hàm do người dùng đặt. Tên hàm được đặt trùng với tên file script

(x1, ..., xM): Các tham số đầu vào của hàm

2. Mô tả

`function [y1,...,yN] = myfun(x1,...,xM)` khai báo một hàm có tên `myfun` chấp nhận đầu vào `x1,...,xM` và trả về đầu ra `y1,...,yN`. Câu lệnh khai báo này phải là dòng thực thi đầu tiên của hàm. Tên hàm không được bắt đầu bằng số và có thể chứa chữ cái, số hoặc dấu gạch dưới.

Cách thức lưu hàm:

- Trong file chứa hàm thì chỉ chứa các định nghĩa hàm. Tên của file phải trùng với tên hàm đầu tiên chứa trong file.
- Trong một file kịch bản (script) chứa các lệnh và định nghĩa hàm, các hàm phải ở cuối file. Tên file script không được trùng tên với một hàm trong file. Hàm được hỗ trợ trong các file script từ phiên bản R2016b trở lên.

Lưu ý: Sử dụng từ khóa `end` để báo hiệu kết thúc một định nghĩa hàm

3. Ví dụ

Hàm với 1 output

Định nghĩa một hàm trong file có tên là computeAverage.m. Hàm này chấp nhận một vector đầu vào, tính giá trị trung bình của các giá trị và trả về một kết quả duy nhất.

```
function ave = calculateAverage(x)
    ave = sum(x(:))/numel(x);
end
```

Gọi hàm

```
z = 1:99;
ave = calculateAverage(z)
```

Kết quả

```
ave =
    50
```

Hàm với nhiều output

Định nghĩa một hàm trong file có tên stat.m trả về giá trị trung bình và độ lệch chuẩn của một vector đầu vào.

```
function [m,s] = stat(x)
    n = length(x);
    m = sum(x)/n;
    s = sqrt(sum((x-m).^2/n));
end
```

Gọi hàm

```
values = [12.7, 45.4, 98.9, 26.6, 53.1];
[ave,stdev] = stat(values)
```

Kết quả

```
ave =
    47.3400
stdev =
    29.4124
```

Hàm chứa trong file script

Định nghĩa và thực thi hàm trong file script sinScript.m để tính giá trị hàm

$$y = \sin^3 x \text{ tại } \frac{2\pi}{3}$$

```
x = 2*pi/3;
y = mySine(x)
```

```
function y = mySine(x)
    y = sin(x).^3;
end
```

Kết quả

```
y =
    0.6495
```

Định nghĩa nhiều hàm trong 1 file

Định nghĩa hai hàm trong một file có tên stat2.m, trong đó hàm đầu tiên gọi hàm thứ hai.

```
function [m,s] = stat2(x)
    n = length(x);
    m = avg(x,n);
    s = sqrt(sum((x-m).^2/n));
end

function m = avg(x,n)
    m = sum(x)/n;
end
```

Hàm avg là một hàm cục bộ. Các hàm cục bộ chỉ khả dụng cho các hàm khác trong cùng một tệp.

Gọi hàm stat2 từ dòng lệnh.

```
values = [12.7, 45.4, 98.9, 26.6, 53.1];
[ave,stdev] = stat2(values)
```

Kết quả

```
ave =
    47.3400
stdev =
    29.4124
```

4. Giá trị trả về của hàm

Mặc định hàm sẽ trả về giá trị được khai báo là output sau khi kết thúc hàm mà không cần từ khóa return. Tuy nhiên, để trả về giá trị và thoát hàm với một điều kiện thì chúng ta sử dụng từ khóa return.

Ví dụ:

Hàm `findSqrRootIndex` tìm chỉ mục của lần xuất hiện đầu tiên của căn bậc hai của một giá trị trong một mảng. Nếu không tìm thấy căn bậc hai, hàm sẽ trả về NaN.

```
function idx = findSqrRootIndex(target,arrayToSearch)

idx = NaN;

if target < 0

    return

end

for idx = 1:length(arrayToSearch)

    if arrayToSearch(idx) == sqrt(target)

        return

    end

end
```

D. THỰC HÀNH

Bài 1

- a. Viết hàm để tính điện trở tương đương của của 1 mạch gồm N điện trở mắc nối tiếp

```
function req = equivRS( r )
```

- b. Áp dụng tính điện trở tương đương của 2 mạch r nối tiếp sau:

$R1 = [10 \ 20 \ 15 \ 16 \ 5]$ và $R2 = [80 \ 200 \ 400 \ 50 \ 250 \ 300]$

Bài 2

Viết hàm tính điện trở tương đương của mạch gồm N điện trở mắc song song. Áp dụng tính điện trở tương đương lần lượt của 2 mạch gồm các điện trở như sau

$R1 = [10 \ 20 \ 15 \ 16 \ 5]$ và $R2 = [80 \ 200 \ 400 \ 50 \ 250 \ 300]$

Bài 3

- a. Viết hàm tính biên độ và vẽ biểu đồ tổng trở của một mạch điện R, L, C. Biết

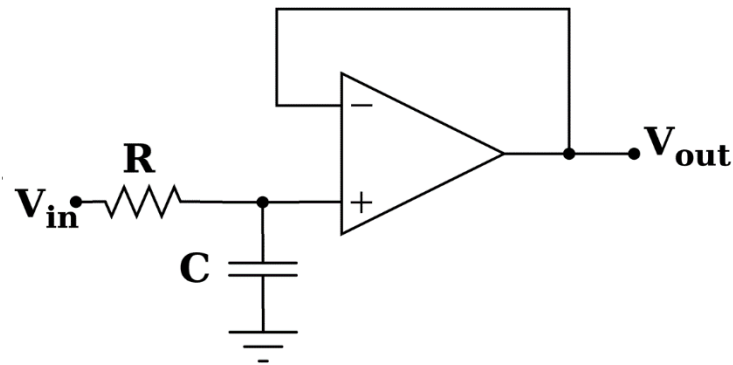
$$Z_{eq} = R + j\omega L + \frac{1}{j\omega C}$$

j là số phức, ω có giá trị từ 0 đến π

- b. Áp dụng đối với $R = 100, L = 4H, C = 1\mu F$

Bài 4

Cho mạch lọc tích cực như hình vẽ. Cho $R = 10k\Omega, C = 100nF$



- a. Viết hàm để vẽ tỷ số $\left| \frac{V_{out}}{V_{in}} \right|$ theo ω , với $\omega = 0:0.01:\pi$.

function Gain(R, C)

- b. Nhận xét về V_{out} khi ω tăng dần

Hướng dẫn:

Ta có: $V_{out} = V_+$ (Mạch đệm). Suy ra

$$V_{out} = V_+ = \frac{Z_C}{R + Z_C} V_{in}$$

$$\frac{V_{out}}{V_{in}} = \frac{Z_C}{R + Z_C} = \frac{\frac{1}{j\omega C}}{R + \frac{1}{j\omega C}} = \frac{1}{j\omega RC + 1}$$

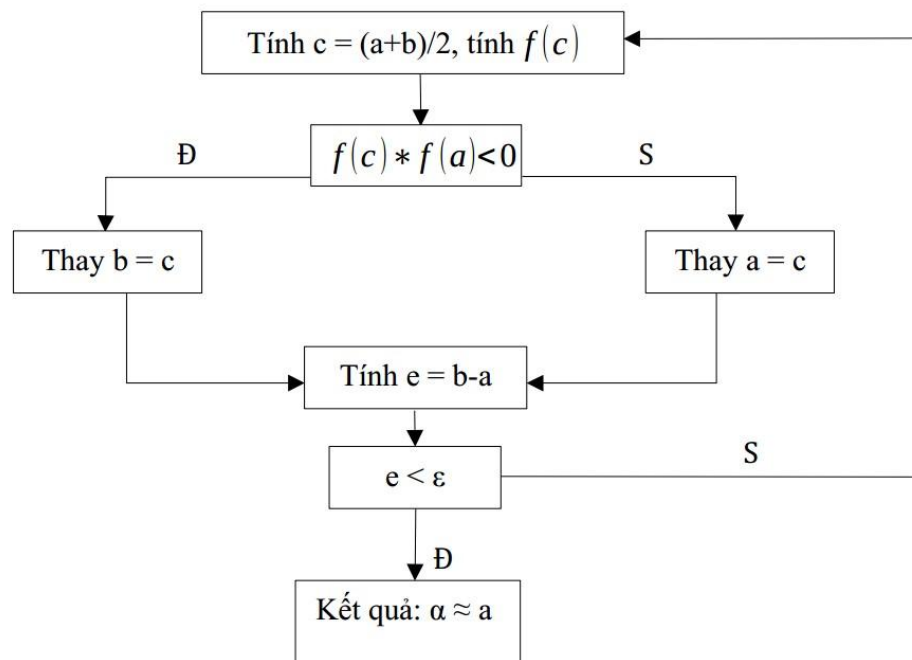
Bài 3: TÍNH GẦN ĐÚNG NGHIỆM PHƯƠNG TRÌNH

A. LÝ THUYẾT

1. Phương pháp chia đôi

- i. Cho phương trình $f(x) = 0$.
- ii. Ấn định sai số cho phép ϵ .
- iii. Xác định khoảng phân li nghiệm $[a, b]$.
- iv. Áp dụng thuật toán trong

Hình 1.



Hình 1: Thuật toán tìm gần đúng nghiệm theo phương pháp chia đôi

2. Phương pháp lặp

- i. Cho phương trình $f(x) = 0$.
- ii. Ấn định sai số cho phép ϵ .
- iii. Xác định khoảng phân li nghiệm $[a, b]$.
- iv. Tìm hàm lặp hội tụ ϕ
- v. Chọn xấp xỉ đầu x_0

Xác định dựa vào cách thu hẹp khoảng phân ly nghiệm như phương pháp chia đôi.

vi. Tính $x_n = \phi(x_{n-1}), n = 1, 2, 3, \dots$

Cho tới khi $|x_n - x_{n-1}| < \epsilon$ thì dừng

3. Phương pháp tiếp tuyến

i. Cho phương trình $f(x) = 0$.

ii. Ấn định sai số cho phép ϵ .

iii. Xác định khoảng phân li nghiệm $[a, b]$.

iv. Chọn xấp xỉ đầu x_0 để $f(x_0) * f'(x_0) > 0$

v. Tính

$$x_n = x_{n-1} - \frac{f(x_{n-1})}{f'(x_{n-1})}$$

Cho tới khi $|x_n - x_{n-1}| < \epsilon$ thì dừng

4. Phương pháp dây cung

i. Cho phương trình $f(x) = 0$.

ii. Ấn định sai số cho phép ϵ .

iii. Xác định khoảng phân li nghiệm $[a, b]$.

iv. Áp dụng thuật toán trong Hình 1 nhưng với

$$c = \frac{af(b) - bf(a)}{f(b) - f(a)}$$

và điều kiện dừng $|c_n - c_{n-1}| < \epsilon$

B. THỰC HÀNH

Bài 1

Sử dụng hàm `syms` để tạo phương trình $fx = x^2 - 2\sin(x) + 1/2$ dưới dạng symbol trong Matlab. Dùng lệnh `fxi = matlabFunction(fx)` để chuyển phương trình sang dạng hàm. Tính $fxi(2)$.

Ví dụ sau minh họa tính $f(x=2)$ của hàm $fex = x - 2$

```
syms x;
fex = x - 2;
fexi = matlabFunction(fex);
fexi(2)
```

Chúng ta cũng có thể sử dụng hàm mạo danh trong Matlab để thực hiện tạo hàm. Ví dụ:

$$fxi = @(x)x^2 + 1$$

hoặc dùng hàm `str2func` để chuyển chuỗi sang hàm:

$$fxi = \text{str2func}('@(x)x^2 + 1')$$

Bài 2

Viết hàm (function) trong Matlab để thực hiện các phương pháp chia đôi, lặp, tiếp tuyến (Newton) và dây cung với định dạng tín hiệu vào và ra như sau:

Phương pháp chia đôi: `function [c,n] = chiadoi(fx, a, b, saiso)`

Phương pháp lặp: `function [x1,n] = lap(fx, fp, a, b, saiso)`

Phương pháp tiếp tuyến: `function [x1,n] = tieptuyen(fx, a, b, saiso)`

Phương pháp dây cung: `function [x1,n] = daycung(fx, a, b, saiso)`

Trong đó:

- f_x là hàm cần tìm gần đúng.
- f_p là hàm lặp.
- a, b là khoảng phân li nghiệm.
- $saiso$ là sai số cho phép.
- n là số lần lặp của phép tính.

a. Tìm nghiệm gần đúng của phương trình $x - \sin x = 0.25$ với khoảng phân li nghiệm $[-1, 2]$ và sai số không lớn hơn 0.005. Cho biết phương pháp nào có số lần lặp nhiều nhất và giải thích:

.....
.....

b. Lặp lại câu a với khoảng phân li nghiệm $[1, 2]$. Cho biết phương pháp nào có số lần lặp ít nhất và giải thích.

.....
.....

c. Dùng phương pháp chia đôi, dây cung, và tiếp tuyến để tính gần đúng nghiệm của phương trình $x^2 - \sin \pi x = 0$ với sai số nhỏ hơn $5 \cdot 10^{-5}$ và khoảng phân li nghiệm lần lượt là $[-0.5, 0.5]$ và $[-0.05, 0.5]$. Giải thích sự khác biệt giữa 2 trường hợp:

.....
.....

PHỤ LỤC

Các câu lệnh trong Matlab được sử dụng

Lệnh	Miêu tả
<code>sym</code>	Đưa biến về dạng dữ liệu symbol.
<code>syms</code>	Khai báo các biến theo dạng dữ liệu symbol.
<code>str2func</code>	Chuyển chuỗi sang hàm
<code>matlabFunction()</code>	Chuyển kiểu symbol sang hàm
<code>@(x)</code>	Tạo hàm số với biến x
<code>diff(fx)</code>	Tìm đạo hàm của hàm số fx ở dạng dữ liệu symbol.
<code>zeros</code>	Tạo ma trận gồm các phần tử có giá trị là 0.
<code>ones</code>	Tạo ma trận gồm các phần tử có giá trị là 1.
<code>simplify</code>	Đơn giản biểu thức dạng symbol
<code>abs</code>	Lấy giá trị tuyệt đối.
<code>sum(A)</code>	Tính tổng các phần tử trong mảng A.
<code>error(message)</code>	Thông báo lỗi chương trình.
<code>fprintf</code>	Tương tự như hàm fprintf trong C.
<code>isempty(A)</code>	Trả về giá trị logic true nếu mảng A rỗng.
<code>size(A)</code>	Trả về ma trận cho biết số lượng các phần tử trong A.

Bài 4: GIẢI HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

E. LÝ THUYẾT

5. Hệ phương trình tuyến tính

Hệ n phương trình bậc 1 có n ẩn số có dạng:

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1;$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2;$$

....

$$a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n;$$

Đặt

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}, A \text{ là ma trận hệ số}$$

$$b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}, b \text{ là ma trận hệ số vế phải}$$

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, x \text{ là ma trận ẩn số}$$

Ta có thể viết lại hệ phương trình dưới dạng $Ax = b$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

6. Phương pháp Gauss

Giải phương trình $Ax = b$ gồm 2 bước

- Bước 1 (Thuận, Forward Elimination): Khử dần các ẩn

Rút gọn $Ax = b$ về dạng ma trận tam giác trên $Tx = b'$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & b_1 \\ a_{21} & a_{22} & a_{23} & b_2 \\ a_{31} & a_{32} & a_{33} & b_3 \end{bmatrix} \Rightarrow \begin{bmatrix} a_{11} & a_{12} & a_{13} & b_1 \\ 0 & a'_{22} & a'_{23} & b'_2 \\ 0 & 0 & a''_{33} & b''_3 \end{bmatrix}$$

Code giả của quá trình khử ản

Tính trong 1 hàng $\rightarrow$ chỉ số cột chạy $\rightarrow$ chỉ số hàng chạy

```
Do k = 1 to n-1
```

```
  Do i = k+1 to n
```

```
    Lambda = ai,k / ak,k
```

```
    Do j = k+1 to n
```

```
      ai,j = ai,j - Lambda × ak,j
```

```
    End Do
```

```
    bi = bi - Lambda × bk
```

```
  End Do
```

```
End Do
```

- Bước 2 (Ngược, Back Substitution): Thay ngược để tìm ản

Giải $Tx = b'$ để tìm x

$$x_3 = \frac{b''_3}{a''_{33}}; \quad x_2 = \frac{b'_2 - a'_{23}x_3}{a'_{22}}; \quad x_1 = \frac{b_1 - a_{13}x_3 - a_{12}x_2}{a_{11}}$$

Code giả của quá trình thế ngược:

```
xn = bn / an,n
```

```
Do k = n-1 down to 1
```

```
  sum = bk
```

```

Do j = k+1 to n
    sum = sum - ak,j * xj
End Do

xk = sum / ak,k

End Do

```

7. Phương pháp phân tích LU

Đề giải: $[A]\{x\}=\{b\}$

Tách : $[L][U]=[A] \rightarrow [L][U]\{x\}=\{b\}$

Trong đó: L : Lower Triangular Matrix , U : Upper Triangular Matrix

Xét: $[U]\{x\}=\{d\}$

$$[L]\{d\}=\{b\}$$

- Giải $[L]\{d\}=\{b\}$ để tìm $\{d\}$
- Giải $[U]\{x\}=\{d\}$ để tìm $\{x\}$

$$[A] = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \Rightarrow A = [L][U] = \begin{bmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

Code Matlab cho quá trình phân tách ma trận A thành LU:

```

function A = LUdec(A)

% LU decomposition of matrix A; returns A = [L\U].

% USAGE: A = LUdec(A)

n = size(A,1);

for k = 1:n-1

```

```
    for i = k+1:n
        if A(i,k) ~= 0.0
            lambda = A(i,k)/A(k,k);
            A(i,k+1:n) = A(i,k+1:n) - lambda*A(k,k+1:n);
            A(i,k) = lambda;
        end
    end
end
```

Hàm tìm nghiệm hệ phương trình bằng phương pháp phân tách LU

```
function x = LUsol(A,b)
% Solves L*U*b = x, where A contains both L and U;
% that is A has the form [L\U].
% USAGE: x = LUsol(A,b)
if size(b,2) > 1; b = b'; end
n = length(b);
for k = 2:n
    b(k) = b(k) - A(k,1:k-1)*b(1:k-1);
end
for k = n:-1:1
    b(k) = (b(k) - A(k,k+1:n)*b(k+1:n))/A(k,k);
end
x = b;
```

F. THỰC HÀNH**Bài 1**

- c. Dựa vào thuật toán ở phần lý thuyết, hoàn thành hàm giải hệ phương trình bằng phương pháp Gauss

`function x = Gauss(A,b)`

- d. Áp dụng hàm Gauss để tìm đáp án cho bài 4 của bài Lab 1.

Bài 2

- a. Dựa vào code Matlab ở phần lý thuyết giải nghiệm hệ phương trình bằng phương pháp phân tích LU. Hãy viết 1 thuật toán khác để thực hiện tách thành ma trận LU và giải nghiệm
- b. Áp dụng để tìm án cho bài 5 của bài Lab 1.

Bài 5: ĐA THỨC NỘI SUY

A. LÝ THUYẾT

1. Đa thức nội suy Lagrange

x	x_0	x_1	x_2	$\dots$	x_{n-1}
y	y_0	y_1	y_2	$\dots$	y_{n-1}

Đa thức nội suy: $p(x) = y_0 l_0(x) + y_1 l_1(x) + y_2 l_2(x) + \dots + y_{n-1} l_{n-1}(x)$

$$l_0 = \frac{(x - x_1)(x - x_2) \dots (x - x_{n-1})(x - x_n)}{(x_0 - x_1)(x_0 - x_2) \dots (x_0 - x_{n-1})(x_0 - x_n)}$$

$$l_{n-1} = \frac{(x - x_0)(x - x_1)(x - x_2) \dots (x - x_{n-2})(x - x_n)}{(x_{n-1} - x_0)(x_{n-1} - x_1)(x_{n-1} - x_2) \dots (x_{n-1} - x_{n-2})(x_{n-1} - x_n)}$$

Tổng quát

$$p_n(x) = \sum_{i=0}^{n-1} L_i(x) f(x_i)$$

với

tích các giá trị

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{x - x_j}{x_i - x_j}$$

Thuật toán:

Input: xa, ya là mảng dữ liệu; x - giá trị cần nội suy

Output: result - kết quả nội suy đa thức Lagrange

```
FUNCTION result = Lagrange(xa, ya, x)
    n = length(xa)
    sum = 0
    FOR i = 1: n DO
```



```

        product = ya_i
    FOR j = 1, n DO
        IF i ≠ j THEN
            product = product*(x - xa_j)/(xa_i - xa_j)
        END IF
    END FOR
    sum = sum + product
END FOR
result = sum
END FUNCTION

```

2. Đa thức Newton

Tỉ hiệu cấp một của y tại x_i và x_j là: $y[x_i, x_j] = \frac{y_i - y_j}{x_i - x_j}$.

Tỉ hiệu cấp hai của y tại x_i, x_j, x_k là: $y[x_i, x_j, x_k] = \frac{y[x_i, x_j] - y[x_j, x_k]}{x_i - x_k}$

Tương ứng cho các tỉ hiệu cấp cao hơn.

Các tỉ hiệu có tính đối xứng: $y[x_i, x_j] = y[x_j, x_i]$.

Đa thức Newton tiến xuất phát từ nút x_0 :

$$p_n(x) = y_0 + (x - x_0)y[x_0, x_1] + (x - x_0)(x - x_1)y[x_0, x_1, x_2] + \dots \\ \dots + (x - x_0)(x - x_1) \dots (x - x_{n-1})y[x_0, \dots, x_n]$$

Đa thức Newton lùi xuất phát từ nút x_n :

$$p_n(x) = y_n + (x - x_n)y[x_n, x_{n-1}] + (x - x_n)(x - x_{n-1})y[x_n, x_{n-1}, x_{n-2}] \\ + (x - x_n)(x - x_{n-1}) \dots (x - x_1)y[x_n, \dots, x_0]$$

Để thực hiện nội suy Newton chúng ta chia thành 2 bước

- Tính tỉ hiệu các cấp
- Thực hiện các số hạng dạng Newton

Thuật toán tính tỉ hiệu

Input: 2 mảng xa, ya

Output: mảng d chứa giá trị tỷ hiệu các cấp

```
FUNCTION d = DividedDifference(xa, ya)

    n = length(xa)

    d = ya

    FOR i = 1, n DO

        FOR j = 1, i-1 DO

            d_i=(d_j - d_i)/(xa_j - xa_i)

        END FOR

    END FOR

END FUNCTION
```

Thuật toán tạo các số hạng dạng Newton

Input: xa – mảng các giá trị x đầu vào; da – mảng các hệ số bảng tỷ hiệu; x – giá trị cần nội suy

Output: result – kết quả nội suy Newton

```
FUNCTION result = NewtonForm(xa, da, x)

    n = length(da)
    result = da(n)
    FOR i = n-1:1 DO
        result = result * (x - xa_i) + da_i
    END FOR

END FUNCTION
```

B. THỰC HÀNH

Bài 1:

- a. Viết hàm (function) trong Matlab để tìm giá trị nội suy sử dụng đa thức nội suy Lagrange
- b. Áp dụng để tìm giá trị nội suy tại $x = 0.14$ và $x = 0.46$. Cho biết 2 các giá trị đầu vào cho nội suy: $x = [0.1, 0.2, 0.3, 0.4]$, $y = [0.09983, 0.19867, 0.29552, 0.38942]$.
- c. So sánh sai số giữa kết quả của b với $\sin(0.14)$ và $\sin(0.46)$.
- d. Viết chương trình tìm đa thức nội suy Lagrange sử dụng kiểu dữ liệu syms trong Matlab

Bài 2

- a. Viết hàm Matlab để thực hiện chức năng tính tỷ hiệu các cấp theo thuật toán ở phần lý thuyết

`function d = DividedDifference(xa, ya)`

- b. Viết hàm Matlab để tìm giá trị nội suy bằng đa thức nội suy Newton

`function result = NewtonForm(xa, da, x)`

- c. Kết hợp 2 hàm để được 1 hàm tìm giá trị nội suy Newton

`function result = NewtonInterpolation(xa, ya, x)`

- d. Áp dụng tìm giá trị nội suy tại $x = 0.14$ và $x = 0.16$. Cho biết 2 các giá trị đầu vào cho nội suy: $x = [0.1, 0.2, 0.3, 0.4]$, $y = [0.09983, 0.19867, 0.29552, 0.38942]$. So sánh kết quả nội suy với phương pháp nội suy Lagrange
- e. Tìm biểu thức nội suy Newton sử dụng kiểu dữ liệu syms trong Matlab

Bài 6: HỒI QUY BÌNH PHƯƠNG TỐI THIỂU

A. LÝ THUYẾT

3. Hồi quy tuyến tính

Hồi quy tuyến tính sử dụng mô hình đường thẳng $y = a_0 + a_1x$ để làm khớp các dữ liệu dữ liệu có được.

Phần sai khác (error) giữa mô hình và các giá trị quan sát được được minh họa bằng phương trình

$$e = y - a_0 - a_1x$$

Mục tiêu của việc làm khớp sử dụng tiêu chuẩn bình phương tối thiểu là tổng bình phương các giá trị lỗi (S_r) này đạt giá trị tối thiểu

$$S_r = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_{i(\text{đo được})} - y_{i(\text{mô hình})})^2 = \sum_{i=1}^n (y_i - a_0 - a_1x_i)^2$$

Để tìm giá trị cực tiểu của S_r ta tìm các giá trị đạo hàm riêng và giải hệ phương trình các đạo hàm riêng

$$\begin{cases} \frac{\partial S_r}{\partial a_0} = 0 \\ \frac{\partial S_r}{\partial a_1} = 0 \end{cases} \Leftrightarrow \begin{cases} -2\sum (y_i - a_0 - a_1x_i) = 0 \\ -2\sum [(y_i - a_0 - a_1x_i)x_i] = 0 \end{cases}$$

$$\begin{bmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum y_i x_i \end{bmatrix}$$

Kết quả thu được hệ số của phương trình hồi quy tuyến tính như sau

$$a_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$$

$$a_0 = \bar{y} - a_1 \bar{x}$$

Trong đó: $\bar{x}, \bar{y}$ là giá trị trung bình của mảng x và mảng y tương ứng

Hệ số tương quan r^2

Gọi S_r là tổng bình phương các giá trị lỗi (error) giữa các giá trị đo được và giá trị của mô hình

$$S_r = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_{i(\text{đo được})} - y_{i(\text{mô hình})})^2 = \sum_{i=1}^n (y_i - a_0 - a_1 x_i)^2$$

Gọi S_t là tổng bình phương của các giá trị lỗi giữa các giá trị đo được và giá trị trung bình

$$S_t = \sum_{i=1}^n (y_i - \bar{y})^2$$

Khi đó hệ số tương quan r^2 được tính theo công thức

$$r^2 = \frac{S_t - S_r}{S_t}$$

r^2 thể hiện kết quả của mô hình hồi quy khớp hay chưa khớp. r càng gần 1 thì kết quả hồi quy càng khớp

Thuật toán:

Input: x, y là mảng dữ liệu đầu vào cần hồi quy

Output: a1, a0 là hệ số của phương trình hồi quy $y = a_0 + a_1.x$

Output: r2 là hệ số tương quan r bình phương

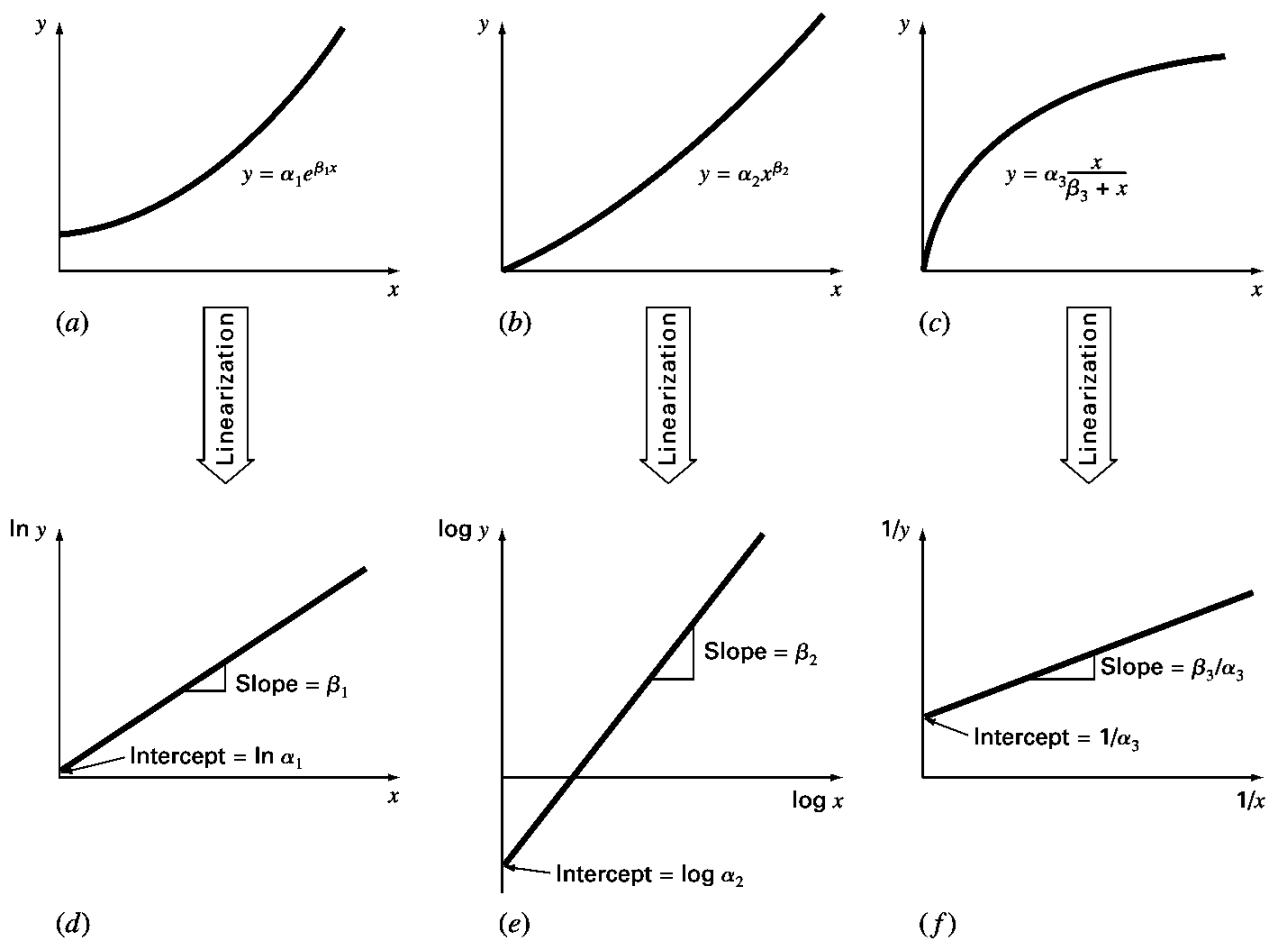
```
function [a1, a0, r2] = Regress(x, y)
    n = length(x)
    sumx = 0
    sumy = 0
    sumxy = 0
```

```
sumx2 = 0
st = 0
sr = 0
for i = 1:n
    sumx = sumx + xi
    sumy = sumy + yi
    sumxy = sumxy + xi*yi
    sumx2 = sumx2 + xi*xi
end for
xm = sumx/n
ym = sumy/n
a1 = (n*sumxy - sumx*sumy)/(n*sumx2 - sumx*sumx)
a0 = ym - a1*xm
for i = 1: n
    st = st + (yi - ym)^2
    sr = sr + (yi - a1*xi - a0)^2
end for
r2 = (st - sr)/st
end function
```

4. Hồi quy phi tuyến

Để hồi quy không tuyến tính, tùy theo dạng dữ liệu ta tuyến tính hóa sau đó thực hiện hồi quy tuyến tính. Cuối cùng suy ra được các hệ số của phương trình phi tuyến

Ví dụ về tuyến tính hóa một số dạng hàm

**Dạng $y = ae^{bx}$**

Lấy Logarit cơ số e 2 vế: $\ln y = \ln a + bx$

Đặt $Y = \ln y$; $A_0 = \ln a$; $A_1 = b$; $X = x$;

Đưa về dạng $Y = A_0 + A_1 X$

Giải hệ phương trình tìm được $A_0, A_1 \Rightarrow a = e^{A_0}$; $b = A_1$

Dạng $y = ax^b$

Lấy Logarit cơ số 10 2 vế: $\lg y = \lg a + b \lg x$

Đặt $Y = \lg y$; $A_0 = \lg a$; $A_1 = b$; $X = \lg x$;

Đưa về dạng $Y = A_0 + A_1X$

Giải hệ phương trình tìm được $A_0, A_1 \Rightarrow a = 10^{A_0}; b = A_1$

Dạng $y = a \frac{x}{b+x}$

Nghịch đảo 2 vế:

$$\frac{1}{y} = \frac{b}{a} \cdot \frac{1}{x} + \frac{1}{a}$$

Đặt $Y = \frac{1}{y}; A_1 = \frac{b}{a}; A_0 = \frac{1}{a}; X = \frac{1}{x};$

Đưa về dạng $Y = A_0 + A_1X$

Giải hệ phương trình tìm được A, B $\Rightarrow a = \frac{1}{A_0}; b = A_1 \cdot a$

B. THỰC HÀNH

Bài 1:

a. Viết hàm tìm mô hình hồi quy tuyến tính ($y = a_0 + a_1x$) trả về hệ số a_0, a_1 và hệ số tương quan r^2

```
function [a1, a0, r2] = HoiQuyTuyenTinh(x, y)
```

b. Áp dụng để tìm mô hình hồi quy cho bảng số liệu sau

x	1	2	3	4	5	6	7	8	9
y	1	1.5	2	3	4	5	8	10	13

Cho biết mức độ khớp dữ liệu của mô hình hồi quy của bảng số liệu cao hay thấp?

c. Vẽ đường thẳng hồi quy và các giá trị đo được lên cùng 1 hình.

Bài 2

- a. Tìm mô hình hồi quy hàm mũ $y = ax^b$ cho bảng số liệu sau đây sử dụng hàm hồi quy tuyến tính ở bài 1.

x	2.5	3.5	5	6	7.5	10	12.5	15	17.5	20
y	13	11	8.5	8.2	7	6.2	5.2	4.8	4.6	4.3

- b. Vẽ đường thẳng hồi quy và các giá trị đo được lên cùng 1 hình.
- c. Dự đoán giá trị của mô hình tại $x = 9$.

Bài 7: TÍNH GẦN ĐÚNG ĐẠO HÀM VÀ TÍCH PHÂN

A. LÝ THUYẾT

1. Tính gần đúng đạo hàm:

Trong trường hợp có các giá trị x, y nhưng việc xây dựng hàm $y = f(x)$ khó khăn hoặc việc tính $y' = f'(x)$ không dễ dàng, ta phải tính gần đúng đạo hàm. Ở đây đề cập 2 cách tính gần đúng sau:

1.1. Tính gần đúng đạo hàm nhờ áp dụng công thức Taylor:

Đạo hàm bậc 1 được tính bình thường như sau:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Từ công thức Taylor:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f^{(3)}(x) + \dots$$

Khi $|h|$ bé thì các số hạng cuối ở vế phải rất bé, ta có thể bỏ qua và có gần đúng:

$$f(x+h) \approx f(x) + hf'(x)$$

Như vậy:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Bảng đạo hàm xấp xỉ trung tâm với sai số cắt cụt $O(h^2)$

	$f(x-2h)$	$f(x-h)$	$f(x)$	$f(x+h)$	$f(x+2h)$
$2hf'(x)$		-1	0	1	
$h^2f''(x)$		1	-2	1	
$2h^3f'''(x)$	-1	2	0	-2	1
$h^4f^{(4)}(x)$	1	-4	6	-4	1

Bảng hệ số của biểu thức xấp xỉ đạo hàm theo chiều tiến cắt cụt $O(h)$

	$f(x)$	$f(x+h)$	$f(x+2h)$	$f(x+3h)$	$f(x+4h)$
$2hf'(x)$	-1	1			
$h^2f''(x)$	1	-2	1		
$2h^3f'''(x)$	-1	3	-3	1	
$h^4f^{(4)}(x)$	-	-4	6	-4	1

Bảng hệ số của biểu thức xấp xỉ đạo hàm theo chiều lùi cắt cụt $O(h)$

	$f(x-4h)$	$f(x-3h)$	$f(x-2h)$	$f(x-h)$	$f(x)$
$2hf'(x)$				-1	1
$h^2f''(x)$			1	-2	1
$2h^3f'''(x)$		-1	3	-3	1
$h^4f^{(4)}(x)$	1	-4	6	-4	1

Bảng hệ số của biểu thức xấp xỉ đạo hàm theo chiều tiến, sai số cắt cụt $O(h^2)$

	$f(x)$	$f(x+h)$	$f(x+2h)$	$f(x+3h)$	$f(x+4h)$	$f(x+5h)$
$2hf'(x)$	-3	4	-1			
$h^2f''(x)$	2	-5	4	-1		
$2h^3f'''(x)$	-5	18	-24	14	-3	
$h^4f^{(4)}(x)$	3	-14	26	-24	11	-2

Bảng hệ số của biểu thức xấp xỉ đạo hàm theo chiều lùi, sai số cắt cụt $O(h^2)$

	$f(x - 5h)$	$f(x - 4h)$	$f(x - 3h)$	$f(x - 2h)$	$f(x - h)$	$f(x)$
$2hf'(x)$				1	-4	3
$h^2 f''(x)$			-1	4	-5	2
$2h^3 f'''(x)$		3	-14	24	-18	5
$h^4 f^{(4)}(x)$	-2	11	-24	26	-14	3

1.2. Tính gần đúng đạo hàm nhờ áp dụng đa thức nội suy:

$$f(x) \approx p_n(x)$$

$$\rightarrow f'(x) \approx p'_n(x)$$

2. Tính gần đúng tích phân

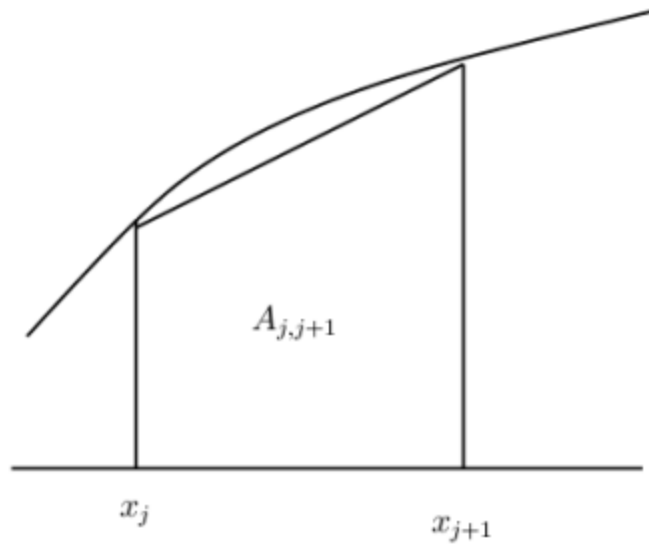
Tính tích phân xác định bình thường như sau:

$$S = \int_a^b f(x)dx$$

Nhưng nếu việc tìm nguyên hàm của $f(x)$ quá khó khăn thì việc tính tích phân phải làm gần đúng.

2.1. Tính gần đúng tích phân với công thức hình thang:

Ý tưởng: chia nhỏ khoảng lấy tích phân $[a, b]$ thành các hình thang. Hình thang cong được thay thế gần đúng bởi hình thang. Như vậy, tích phân gần đúng là tổng các diện tích hình thang nhỏ. Cách này tương đương với việc lấy tích phân của hàm nội suy bậc 1 của phương pháp nội suy Newton tiến với khoảng cách đều.



$$I = \int_a^b f(x)dx \approx \frac{h}{2} \left[\{f(a) + f(b)\} + 2 \sum_{i=1}^{N-1} f(a + ih) \right]$$

Công thức hình thang chia khoảng $[a,b]$ thành N đoạn cong bằng nhau:

$$h = \frac{b - a}{N}$$

2.2. Tính gần đúng tích phân với công thức Simpson:

Ý tưởng: do việc tính gần đúng tích phân là việc tích phân của hàm nội suy nên hàm nội suy chính xác hơn cho kết quả gần đúng có sai số nhỏ hơn. Trong công thức Simpson, việc tính gần đúng tích phân là việc tích phân của hàm nội suy bậc 2 của phương pháp nội suy Newton tiến với khoảng cách đều.

$$I = \int_a^b f(x)dx \approx \frac{h}{3} \left[\{f(a) + f(b)\} + 4 \sum_{\substack{i=1 \\ i \text{ lẻ}}}^{N-1} f(a + ih) + 2 \sum_{\substack{i=2 \\ i \text{ chẵn}}}^{N-1} f(a + ih) \right]$$

Công thức Simpson chia đoạn $[a,b]$ thành $2N$ đoạn cong bằng nhau:

$$h = \frac{b - a}{N}$$

B. THỰC HÀNH

Bài 1:

Cho hàm số $y = \arcsin(x)$ với các giá trị tại: $x = [0.1 \ 0.3 \ 0.5 \ 0.7 \ 0.9]$ và $y = [0.1002 \ 0.3047 \ 0.5236 \ 0.7754 \ 1.1198]$. Tính gần đúng đạo hàm của y tại $x = 0.5$ và so sánh với đạo hàm chính xác $\left(\frac{1}{\sqrt{1-x^2}}\right)$

- Áp dụng công thức Taylor
- Đa thức nội suy (sử dụng phương pháp nội suy Lagrange.)

Bài 2

- Viết function tính gần đúng tích phân của một hàm số $f(x)$ bất kỳ trong đoạn $[a, b]$ sử dụng công thức hình thang với N đoạn con bằng nhau.

Function $y = \text{TichPhanHinhThang}(fx, a, b, N)$

- Áp dụng hàm trên để tính gần đúng tích phân của hàm số $f(x) = x^3 \sin(x)$ trong đoạn $[0, 1]$ với $N = 10$. So sánh với kết quả sử dụng hàm `quad` của Matlab.

Bài 3

- Viết function tính gần đúng tích phân của một hàm số $f(x)$ bất kỳ trong đoạn $[a, b]$ sử dụng công thức Simpson với N đoạn con bằng nhau.

Function $y = \text{TichPhanSimpson}(fx, a, b, N)$

- Áp dụng hàm trên tính gần đúng tích phân của hàm số $f(x) = x^3 \sin(x)$ trong đoạn $[0, 1]$ với $N = 10$. So sánh với kết quả sử dụng hàm `quad` của Matlab.

Bài 8:

GIẢI GẦN ĐÚNG NGHIỆM PHƯƠNG TRÌNH VI PHẦN

B. LÝ THUYẾT

3. Phương pháp chuỗi Taylor:

$y(x)$ xấp xỉ bằng tổng N số hạng đầu của chuỗi Taylor. N càng lớn, độ chính xác càng cao.

Tuy nhiên phương pháp này chỉ cho $y(x)$ gần đúng với các x gần. Do vậy, phương pháp này ít được sử dụng và ít được đề cập tới trong các sách

$$y(x) = y(x_0) + \frac{y'(x_0)}{1!}(x - x_0) + \frac{y''(x_0)}{2!}(x - x_0)^2 + \dots + \frac{y^{(k)}(x_0)}{k!}(x - x_0)^k + \dots \quad (1)$$

4. Phương pháp Euler (Ơ-le):

Chia đoạn $[x_0, X]$ thành N đoạn bằng nhau. $h = (X - x_0)/N$.

Mục đích của phương pháp Ơ-le là tính gần đúng giá trị của $y(x)$ chỉ tại một nút x_i chứ không phải tại mọi $x \in [x_0, X]$:

$$y_{i+1} = y_i + hf(x_i, y_i) \quad (2)$$

Để tăng độ chính xác ta giảm bước nhảy h , nhưng nếu $h < h_0$, với h_0 là giá trị h tối ưu của phương pháp Ơ-le, thì sai số lại tăng.

Thuật toán:

```
function [x,y] = Ole(fxy, xdau, xcuo, y0, N)
```

```
h = (xcuo - xdau)/N
```

```
x = xdau:h:xcuo
```

```
y = x;
```

```
y(1) = y0
```

```

    for i = 1:N
         $y_{i+1} = y_i + h * f_{xy}(x_i, y_i)$ 
    end
end

```

5. Phương pháp hiện ẩn hình thang (Heun)

Tương tự phương pháp Ô-le, mục đích của phương pháp hình thang cũng là tính gần đúng giá trị của $y(x)$ chỉ tại một nút x_i chứ không phải tại mọi $x \in [x_0, X]$. Phương pháp hiện ẩn trung điểm có độ chính xác bậc 2 (khi h tương đối bé).

Bước 1: lấy xấp xỉ đầu bằng phương pháp Ô-le

$$y_{i+1}^0 = y_i + hf(x_i, y_i)$$

Bước 2: dùng hàm lặp tính lại các giá trị của y_{i+1}

$$y_{i+1} = y_i + \frac{h}{2} [f(x_i, y_i) + f(x_{i+1}, y_{i+1}^0)]$$

Hàm lặp dừng khi:

$$|y_{i+1}^m - y_{i+1}^{m-1}| \leq \varepsilon$$

Trong đó, ε là sai số cho trước, $m = 1, 2, 3, \dots$

Thuật toán

```

function [x,y] = HienAnHinhThang(fxy, xdau, xcuo, y0, N, e)

    h = (xcuo - xdau)/N

    x = xdau:h:xcuo

    y = x

    y(1) = y0;

```



```

    for i = 1:N

         $y_{i+1} = y_i + h * f_{xy}(x_i, y_i)$ 

        loop

            c =  $y_{i+1}$ 

             $y_{i+1} = y_i + \frac{h}{2} * (f_{xy}(x_i, y_i) + f_{xy}(x_{i+1}, y_{i+1}))$ 

            if abs( $y_{i+1} - c$ ) < e

                break;

            end

        end

    end

end

end

```

6. Phương pháp hiện ẩn trung điểm (Euler cải tiến)

Tương tự phương pháp hiện ẩn hình thang, mục đích của phương pháp hiện ẩn trung điểm cũng là tính gần đúng giá trị của $y(x)$ chỉ tại một nút x_i chứ không phải tại mọi $x \in [x_0, X]$. Phương pháp hiện ẩn trung điểm có độ chính xác bậc 2 (khi h tương đối bé) được tính như sau:

Bước 1: Tính giá trị tại điểm giữa $y_{i+1/2}$:

$$y_{i+1/2} = y_i + \frac{h}{2} f(x_i, y_i)$$

Bước 2:

$$y_{i+1} = y_i + hf(x_i + \frac{h}{2}, y_{i+1/2})$$

Thuật toán

```
function [x,y] = HienAnTrungDiem(fxy, xdau, xcuo, y0, N)
```

```

h = (xcuoi - xdau)/N

x = xdau:h:xcuoi

y = x

y(1) = y0

for i = 1:N

     $y_{i+1} = y_i + \frac{h}{2} * f_{xy}(x_i, y_i)$ 

     $y_{i+1} = y_i + h * f_{xy}\left(x_i + \frac{h}{2}, y_{i+1}\right)$ 

end

end

```

7. Phương pháp Runge-Kutta

Phương pháp R-K có độ chính xác cấp 4.

$$y_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Với:

$$k_1 = hf(x_i, y_i)$$

$$k_2 = hf(x_i + 0.5h, y_i + 0.5k_1)$$

$$k_3 = hf(x_i + 0.5h, y_i + 0.5k_2)$$

$$k_4 = hf(x_i + h, y_i + k_3)$$

Thuật toán

```

function [x,y] = RK(fxy, xdau, xcuoi, y0, N)

h = (xcuoi - xdau)/N

```

```

x = xdau:h:xcuoi
y = x
y(1) = y0;
for i = 1:N
    k1 = h*fxy(xi, yi)
    k2 = h*fxy(xi + h/2, yi + k1/2)
    k3 = h*fxy(xi + h/2, yi + k2/2)
    k4 = h*fxy(xi + h, yi + k3)
    yi+1 = yi + (k1 + 2*k2 + 2*k3 + k4)/6
end
end

```

B. THỰC HÀNH

Sử dụng phương trình vi phân $\frac{dy}{dx} = (1 - x)y = f(x, y)$ cho toàn bài thực hành

Điều kiện ban đầu: $y(0) = 1$

Nghiệm chính xác: $y(x) = e^{x - \frac{x^2}{2}}$

Bài 1

a. Viết function áp dụng phương pháp O-le, tính gần đúng hàm $y(x)$: function [x,y]

```
function [x,y] = Ole(fxy, xdau, xcui, y0, N)
```

- fxy là hàm vi phân $f(x, y)$
- xdau và xcui là giá trị đầu và cuối của mảng x
- y0 là giá trị điều kiện ban đầu
- N là số đoạn được chia

- b. Vẽ đồ thị gần đúng của $y(x)$ với $h = 0.2$ (ứng với $N = 25$) và $0 \leq x \leq 5$

Bài 2

- a. Viết function áp dụng phương pháp hiện ẩn hình thang, tính gần đúng hàm $y(x)$:

```
function [x, y] = HienAnHinhThang(fxy, xdau, xcuo, y0, N, e)
```

- e là sai số cho phép

- b. Vẽ đồ thị gần đúng của $y(x)$ với $h = 0.2$ (ứng với $N = 25$) và $0 \leq x \leq 5$ và $e = 0.001$

Bài 3

- a. Viết function áp dụng phương pháp hiện ẩn trung điểm, tính gần đúng hàm $y(x)$:

```
function [x, y] = HienAnTrungDiem(fxy, xdau, xcuo, y0, N)
```

- b. Vẽ đồ thị gần đúng của $y(x)$ với $h = 0.2$ (ứng với $N = 25$) và $0 \leq x \leq 5$

Bài 4

- a. Viết function áp dụng phương pháp Runge-Kutta, tính gần đúng hàm $y(x)$:

```
function [x,y] = RK(fxy, xdau, xcuo, y0, N)
```

- b. Vẽ đồ thị gần đúng của $y(x)$ với $h = 0.2$ (ứng với $N = 25$) và $0 \leq x \leq 5$

Bài 5

Vẽ chung tất cả các $y(x)$ gần đúng của các phương pháp vào 1 đồ thị và nhận xét độ chính xác giữa các phương pháp với nhau

Bài 9A: MATLAB APP DESIGNER

A. LÝ THUYẾT

Từ phiên bản 2016, Matlab đã sử dụng App Designer để thiết kế giao diện để thay thế cho **GUIDE**. App Designer có giao diện hiện đại, nhiều đối đối hơn, tốc độ thực thi nhanh hơn, cách set và get các thuộc tính giống như các ngôn ngữ hướng đối tượng hiện tại như C#, Java...

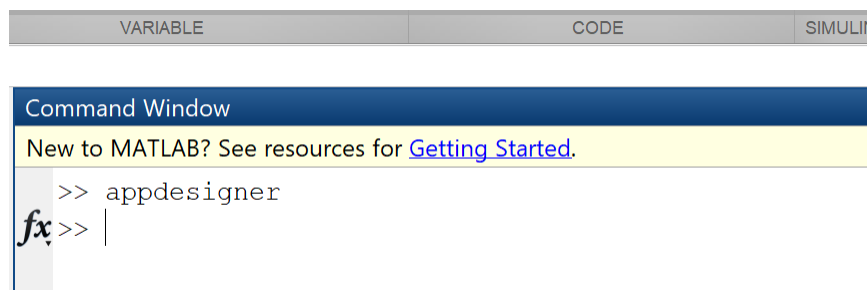
Ví dụ 1: Thiết kế giao diện và viết chương trình để vẽ tín hiệu $s = 2 \sin(2\pi f n)$ với các thông số được nhập vào từ giao diện:

- f: tần số tín hiệu
- fs: tần số lấy mẫu tín hiệu
- t: thời gian vẽ tín hiệu

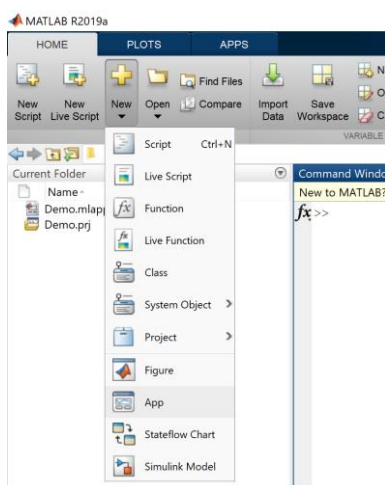
Hướng dẫn:

Bước 1: Khởi động App Designer

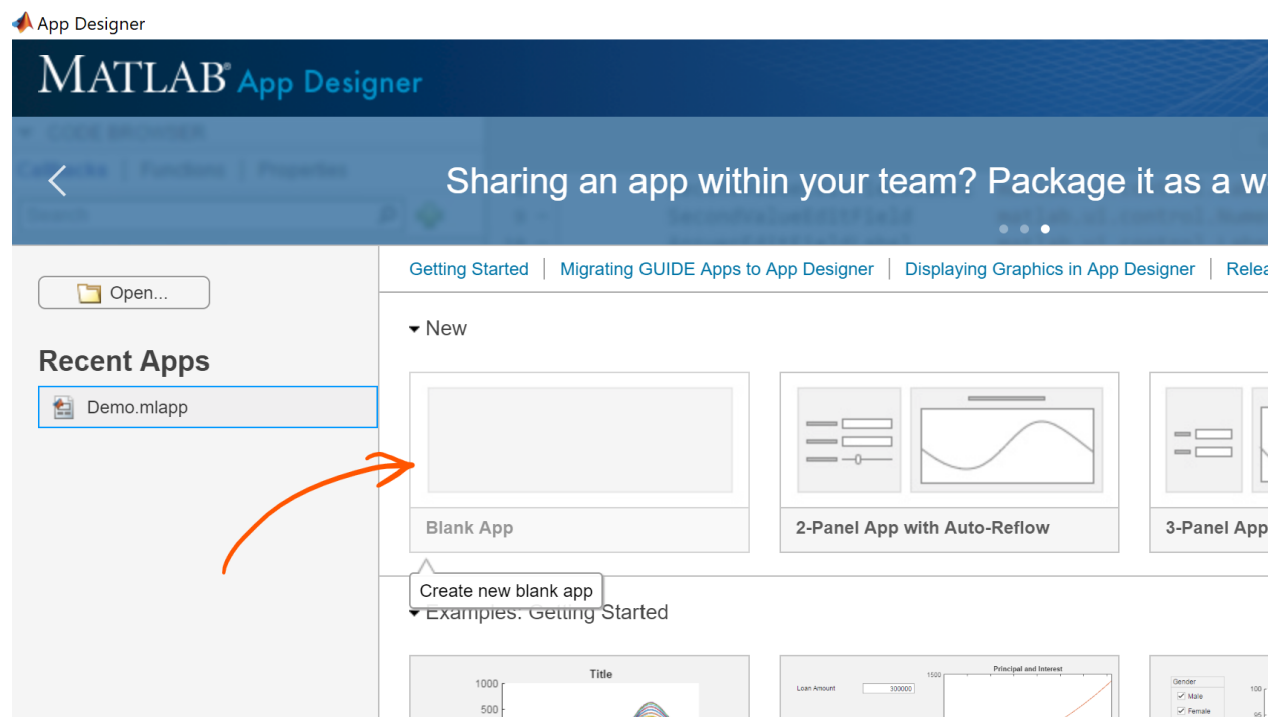
Vào Command windows gõ câu lệnh `appdesigner` và Enter



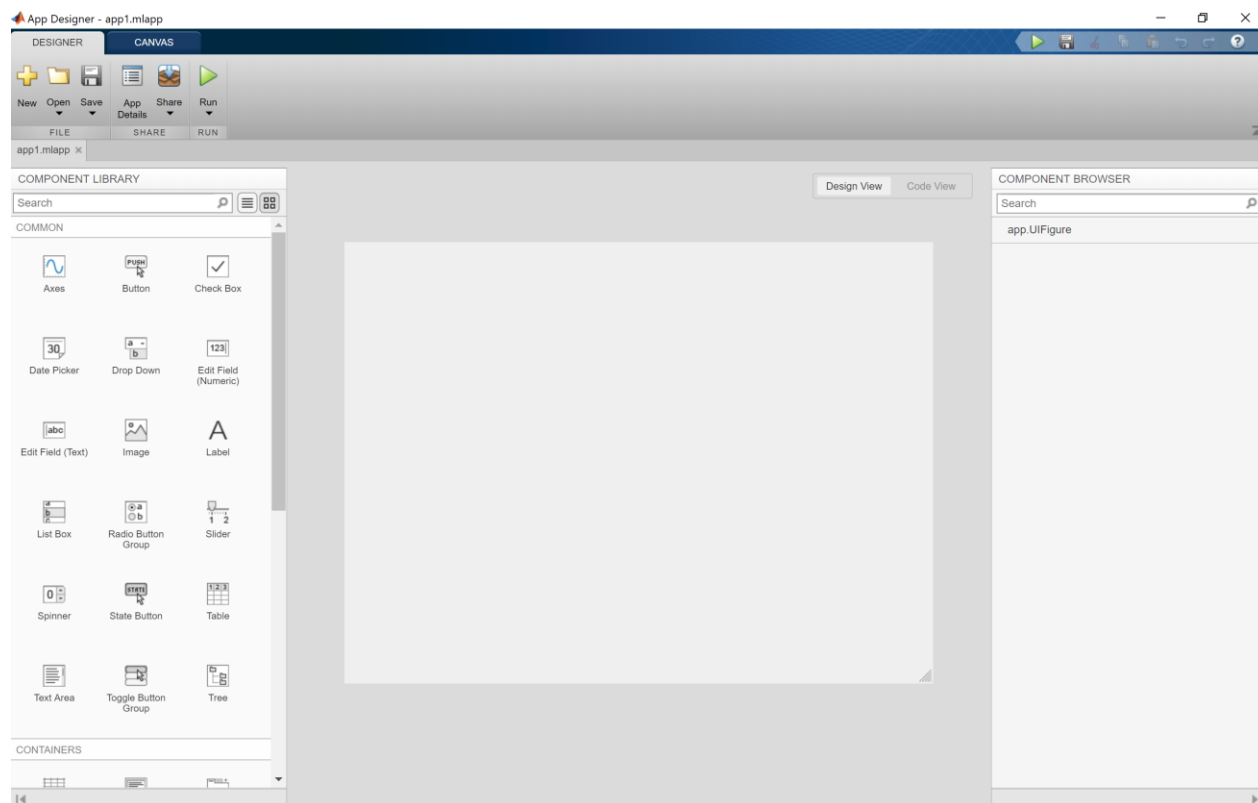
Hoặc vào New → App như hình vẽ



Cửa sổ App Designer xuất hiện → Chọn Blank App



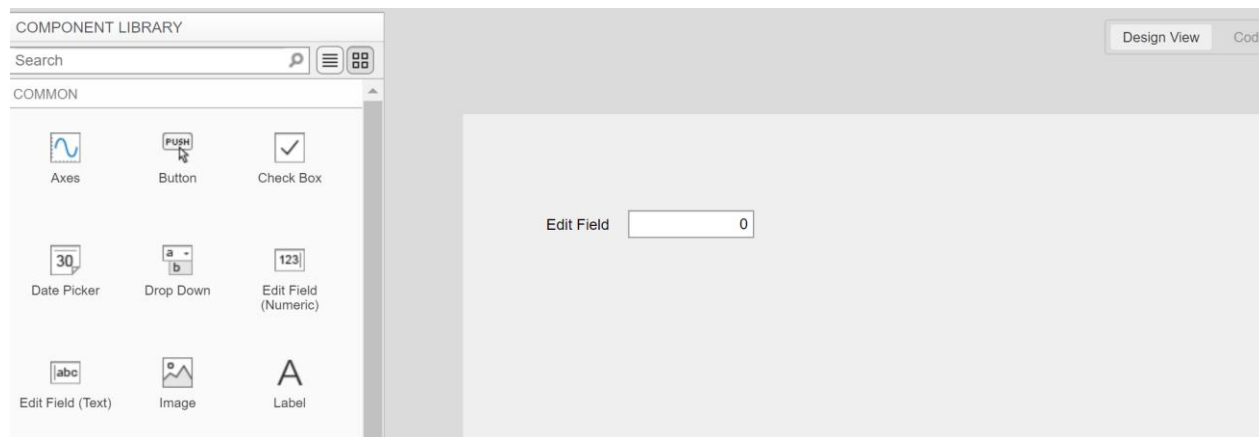
Màn hình thiết kế xuất hiện



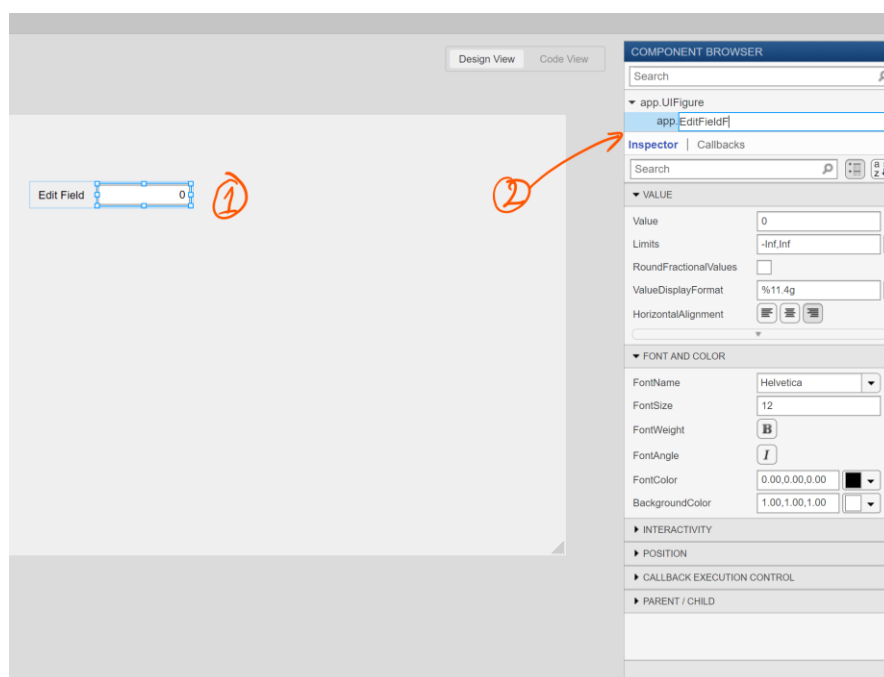
Bước 2: Thêm các đối tượng

*Edit Field (Numeric) để nhập tần số tín hiệu

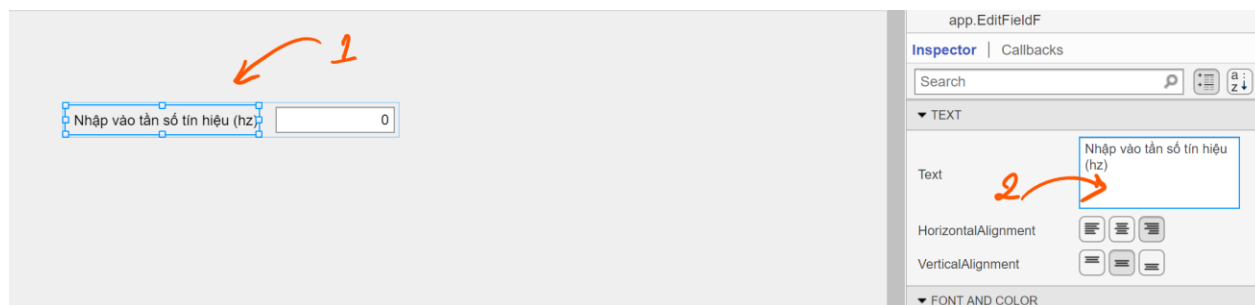
Kéo chuột để lấy đối tượng **Edit Field (Numeric)**



Đặt tên cho đối tượng là *EditFieldF* bằng cách click chuột lên đối tượng và thay đổi tên đối tượng bên cửa sổ Components Browser

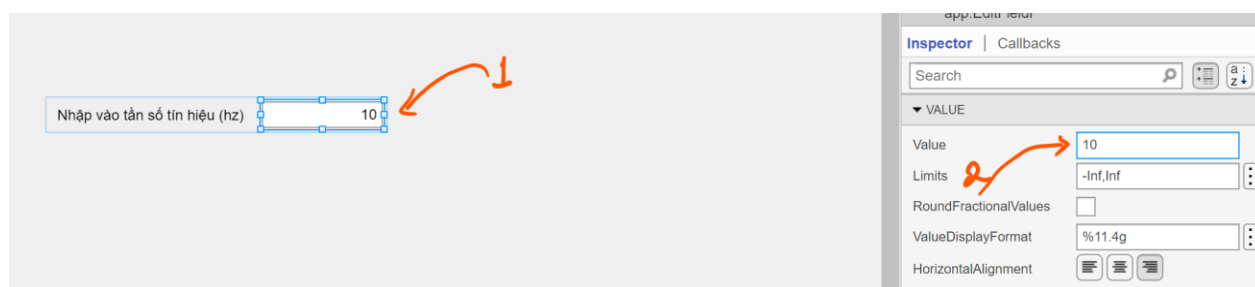


Thay đổi thuộc tính Text cho Edit Field: Click chuột lên Edit Field → Thay đổi giá trị trong Inspector thành “Nhập vào tần số tín hiệu (Hz)”

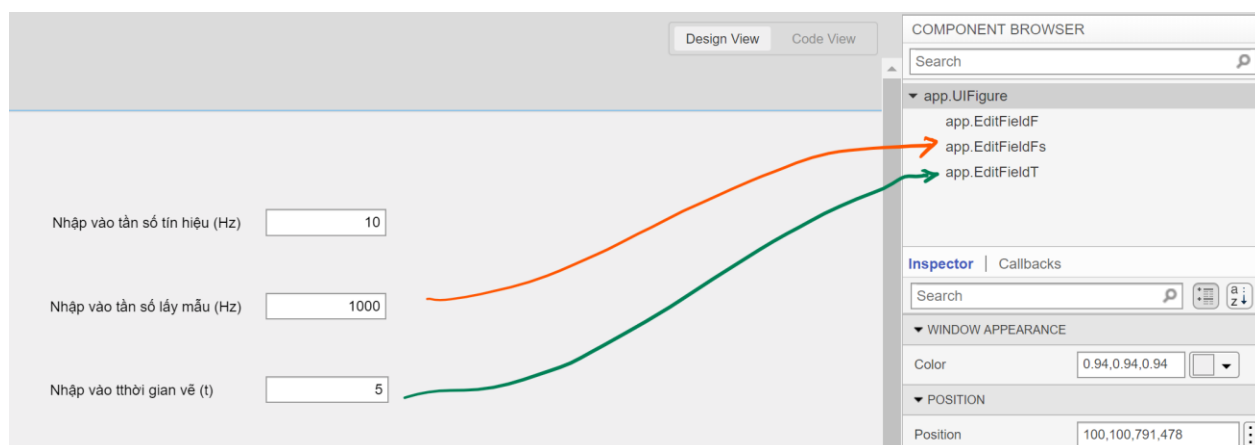


Thay đổi thuộc tính Value của đối tượng sang 10

Lưu ý: Ta có thể double click để sửa giá trị trực tiếp trên giao diện



Tương tự ta làm các đối tượng tần số lấy mẫu và thời gian với tên và các thuộc tính như hình sau

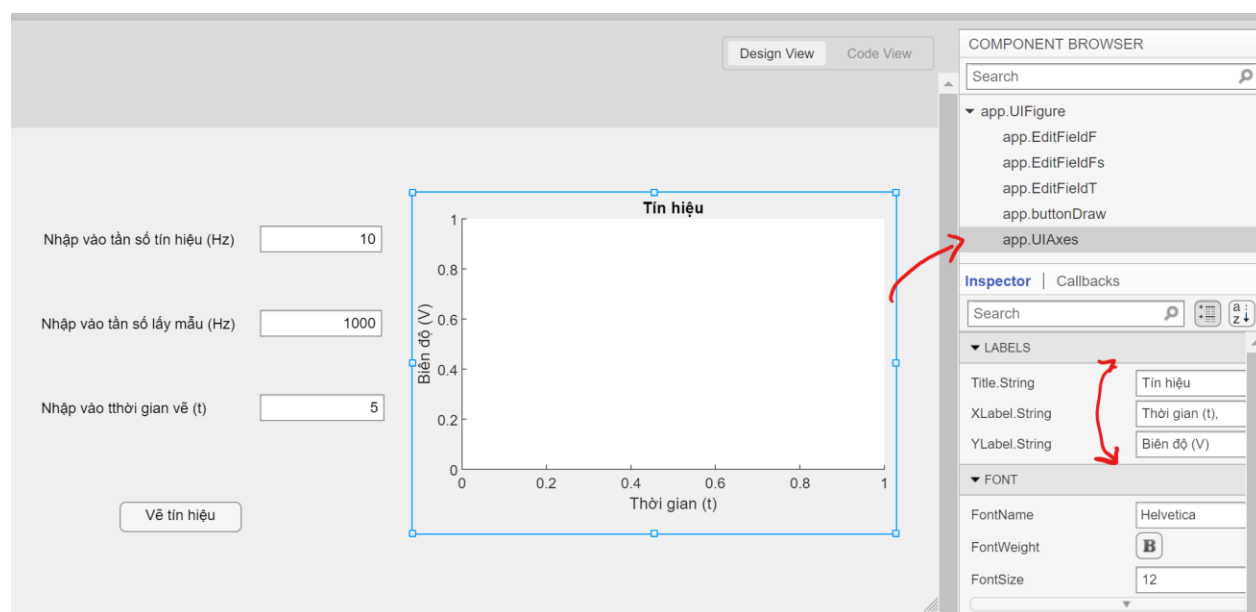


*Thêm nút nhấn và Axes

Thêm đối tượng *Button* với tên là *buttonDraw* và thuộc tính *Text* là “Vẽ tín hiệu”

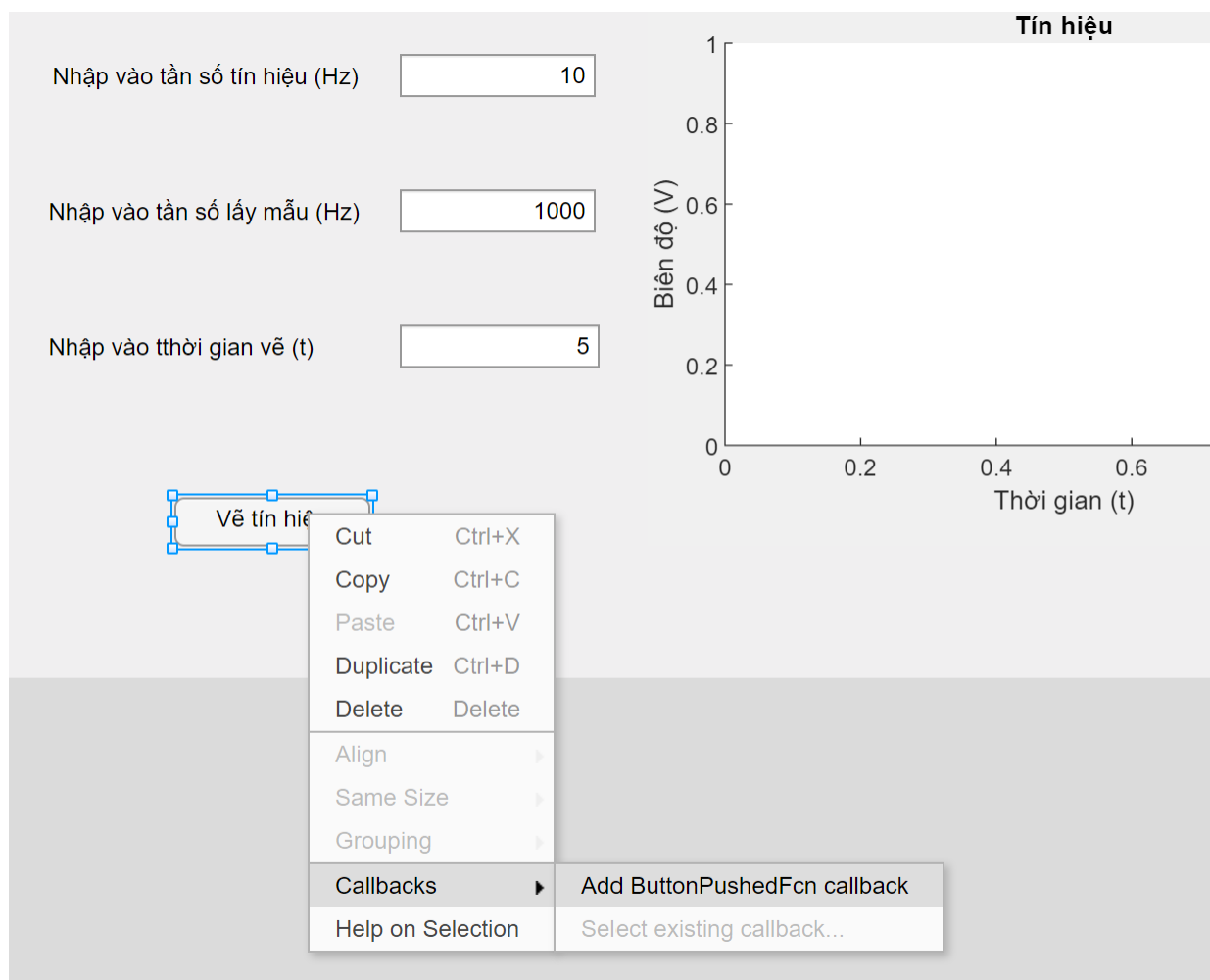


và *Axes* với tên là *UIAxes* và các thuộc tính *Title*, *XLabel*, *YLabel* như hình vẽ

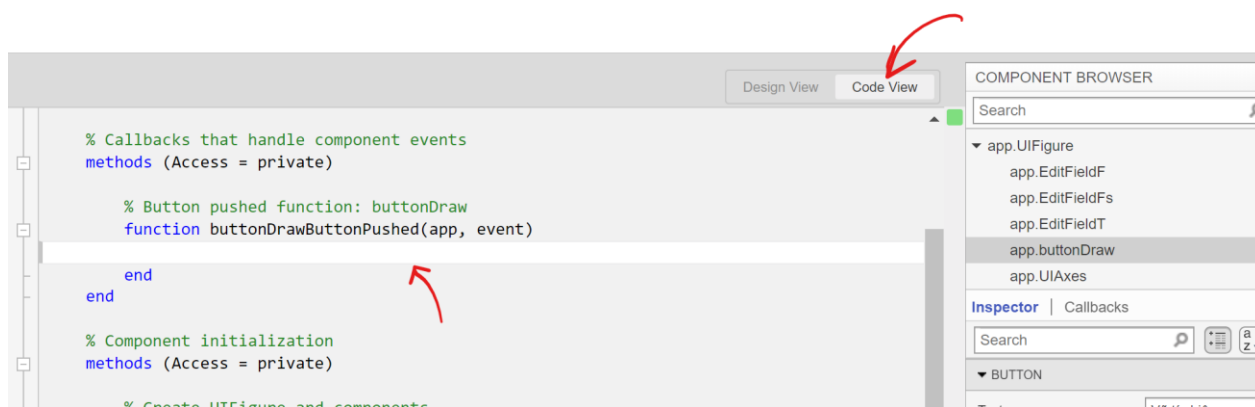


Bước 3: Tạo sự kiện Callback cho nút nhấn Vẽ tín hiệu

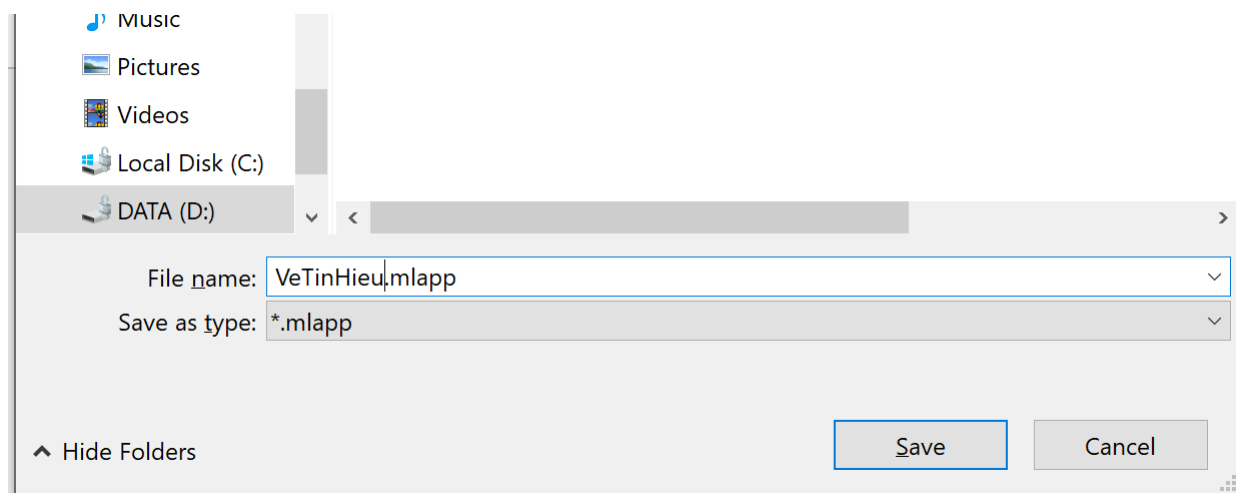
Click chuột phải lên nút nhấn Vẽ tín hiệu -> **View Callbacks** -> **Add ButtonPushedFcn callback** để mở sự kiện khi nút nhấn được nhấn



Cửa sổ *Design View* chuyển sang *Code View*



Lưu lại quá trình làm việc bằng cách nhấn **Ctrl+S**. Đặt tên là *VeTinHieu.mlapp* như hình vẽ và chọn **Save**



Bước 4: Viết chương trình cho sự kiện nhấn nút Vẽ tín hiệu

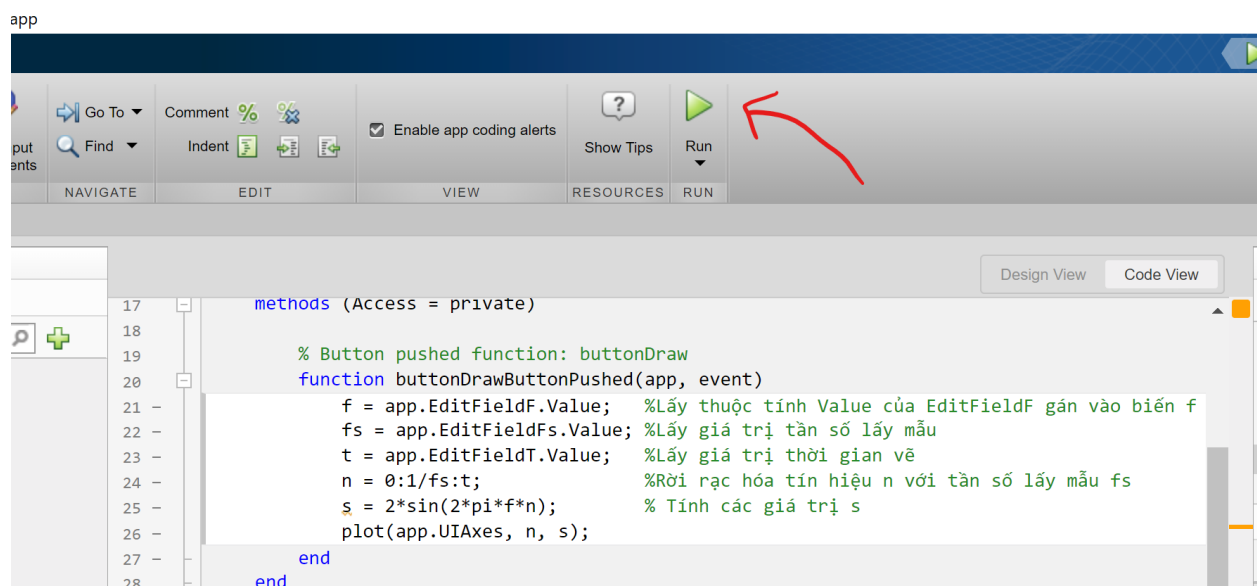
Tại hàm `function buttonDrawButtonPushed(app, event)`

Ta viết đoạn chương trình sau

```
function buttonDrawButtonPushed(app, event)
    f = app.EditFieldF.Value;    %Lấy thuộc tính Value của
    EditFieldF gán vào biến f
    fs = app.EditFieldFs.Value; %Lấy giá trị tần số lấy mẫu
    t = app.EditFieldT.Value;   %Lấy giá trị thời gian vẽ
    n = 0:1/fs:t;               %Rời rạc hóa tín hiệu n với
    tần số lấy mẫu fs
    s = 2*sin(2*pi*f*n);        % Tính các giá trị s
    plot(app.UIAxes, n, s);
end
```

Bước 5: Thực thi chương trình

Vào biểu tượng **Run** để chạy chương trình

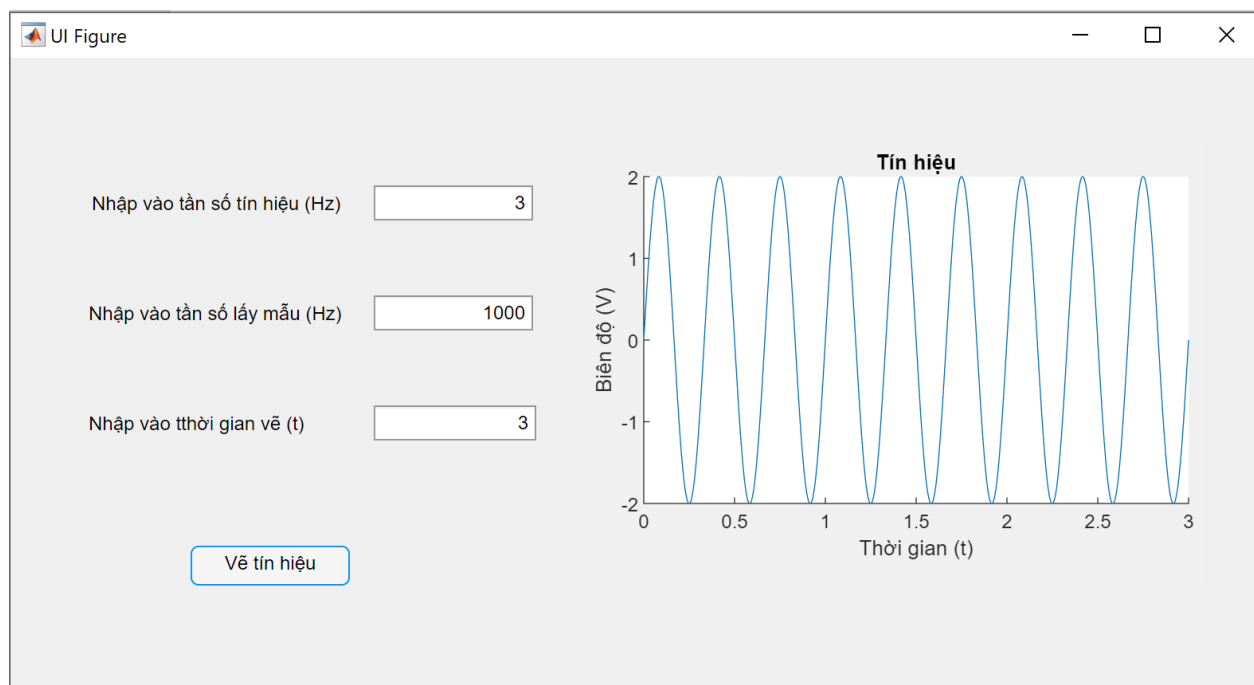


Kết quả chạy chương trình như sau



Chúng ta có thể thay đổi các giá trị tần số tín hiệu, tần số lấy mẫu và thời gian vẽ sau đó nhấn nút Vẽ tín hiệu để xem kết quả

Kết quả với $t = 2$, $f = 3$, $fs = 1000$ như sau

**Ví dụ 2:**

Cải tiến chương trình ở ví dụ 1 để thực hiện chức năng

sau: vẽ tín hiệu của 1 hàm bất kỳ theo x với

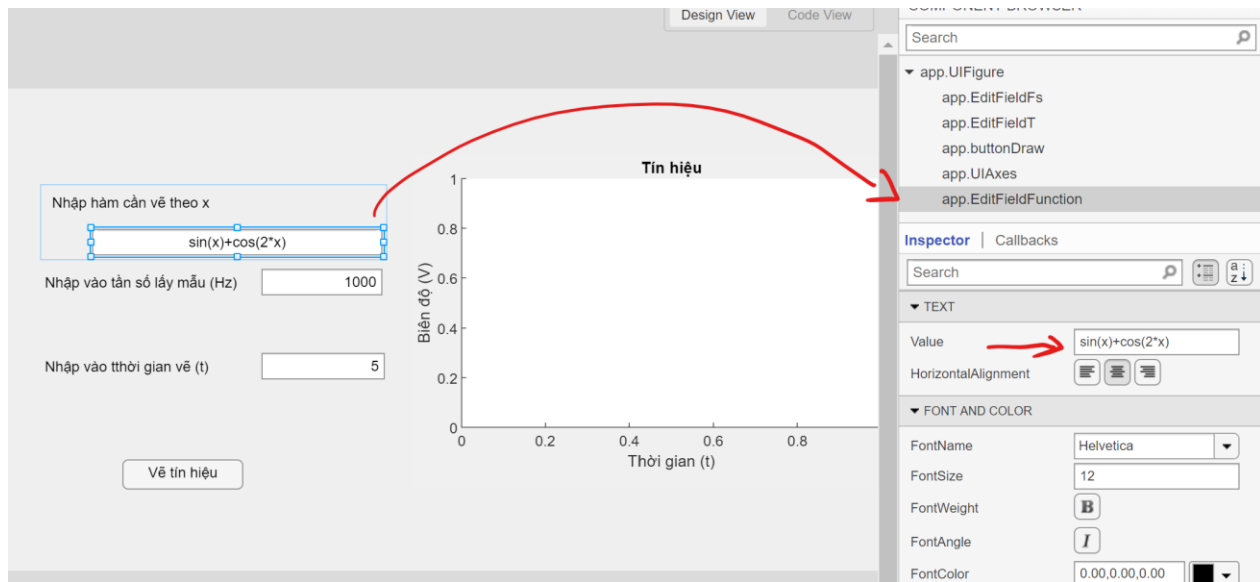
-Hàm nhập vào từ Edit Field Text

-Thời gian vẽ t

-Tần số lấy mẫu fs

Hướng dẫn:

Tạo giao diện như sau



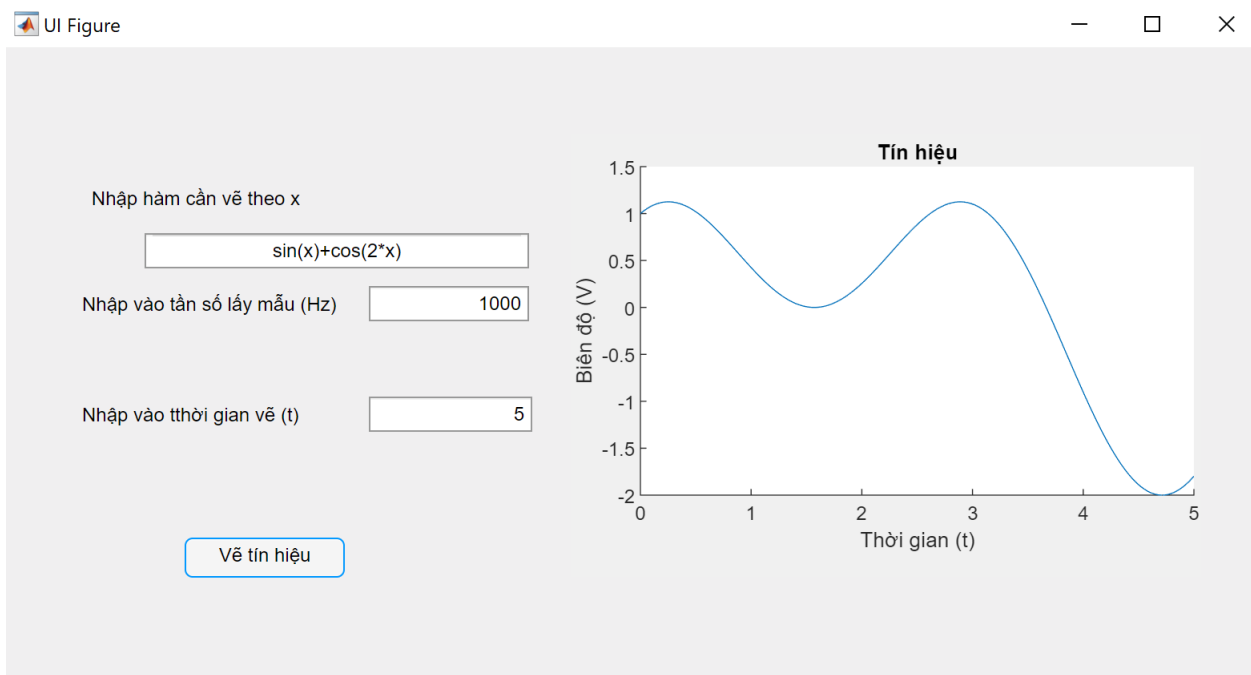
Thuộc tính của các phần tử giống ví dụ 1 đều giữ nguyên

Thêm phần tử Edit Field (Text) để nhập hàm với tên và các thuộc tính như hình

Trong hàm `buttonDrawButtonPushed` chỉnh sửa lại đoạn chương trình như sau:

```
function buttonDrawButtonPushed(app, event)
    fs = app.EditFieldFs.Value; %Lấy giá trị tần số lấy mẫu
    t = app.EditFieldT.Value;   %Lấy giá trị thời gian vẽ
    ham = app.EditFieldFunction.Value;
    ham = str2func(['@(x)',ham]);
    x = 0:1/fs:t;               %Rời rạc hóa tín hiệu x với tần số lấy
    mẫu fs                      %Tính các giá trị y theo x
    y = ham(x);
    plot(app.UIAxes, x, y);
end
```

Kết quả như sau:



Bài 9B: MATLAB APP DESIGNER

B. THỰC HÀNH

Bài 1

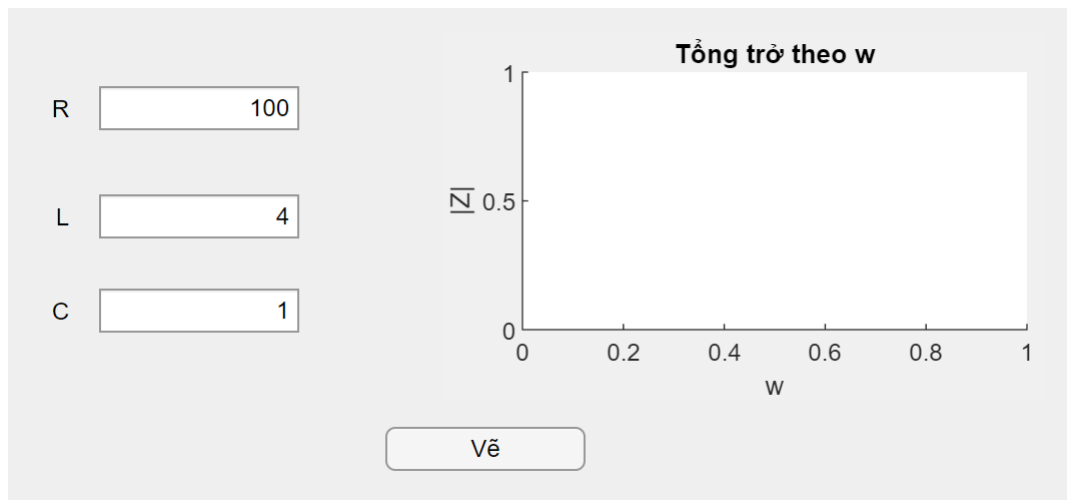
Cho mạch điện R, L, C mắc nối tiếp với tổng trở tương đương Z như sau

$$Z(j\omega) = R + j\omega L + \frac{1}{j\omega C}$$

R, L, C được nhập vào từ giao diện

Vẽ giá trị độ lớn của tổng trở $|Z(j\omega)|$ theo tần số góc ω (ω có giá trị từ 0 đến 2π) với các giá trị R, L, C.

Biết R (Ω), L (H), C (μF)



Bài 2

Cho dòng điện I_{DS} của một MOSFET kênh N được cho như sau

Vùng	Điều kiện	Dòng I_D
Ngưng dẫn (Cut-off)	$V_{GS} \leq V_T$	$I_{DS} = 0$
Tuyến tính (Linear)	$V_{GS} > V_T, V_{DS} \leq V_{GS} - V_T$	$I_{DS} = \mu_n C_{ox} \frac{W}{L} \left[(V_{GS} - V_T)V_{DS} - \frac{V_{DS}^2}{2} \right] (1 + \lambda V_{DS})$
Bão hòa (Saturation)	$V_{GS} > V_T, V_{DS} > V_{GS} - V_T$	$I_{DS} = \mu_n C_{ox} \frac{W}{L} (V_{GS} - V_T)^2 (1 + \lambda V_{DS})$

Vẽ đặc tính của MOSFET với các thông số nhập từ giao diện

Tham số	Đơn vị	Giá trị thông thường
$\mu_n C_{ox}$	$\frac{\mu A}{V^2}$	200
V_T	V	0.5-1.0
λ	V^{-1}	0.005-0.05
W, L	μm	90-180

Cho V_{DS} có giá trị từ 0 đến 5 bước là 0.1.

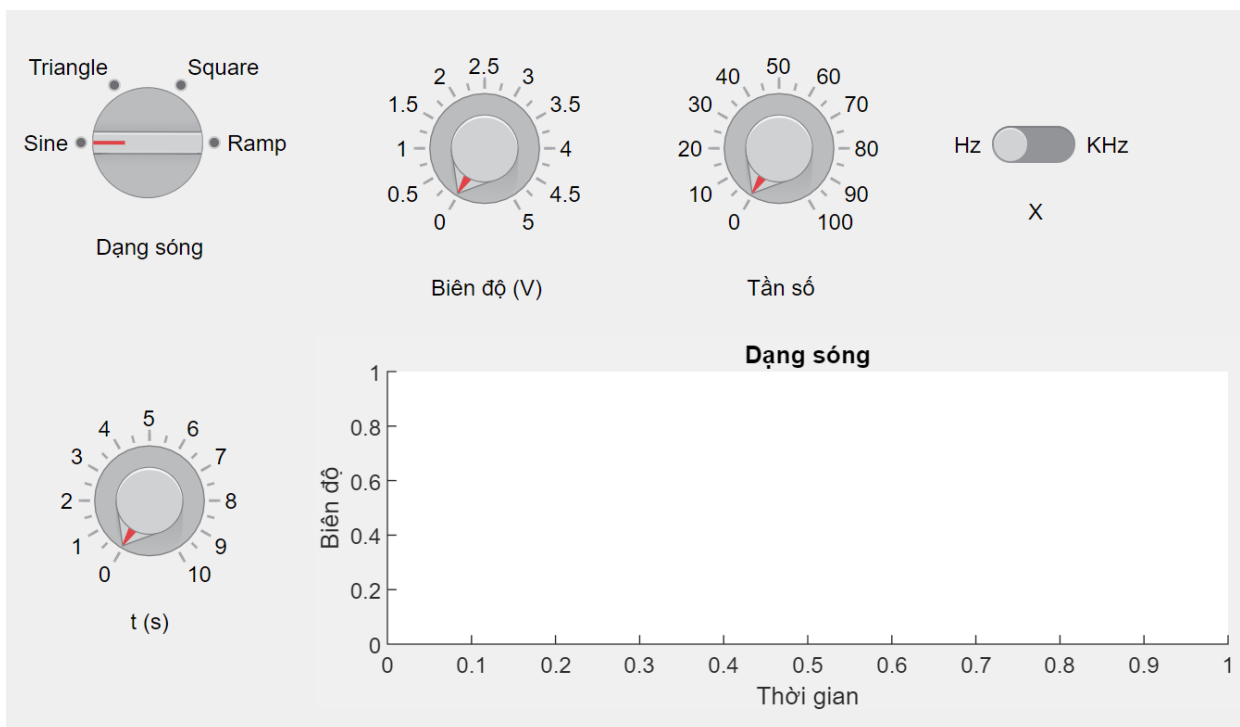
Vẽ đặc tuyến của MOSFET trên cùng biểu đồ với V_{GS} lần lượt là $V_{GS1} = 0.5, V_{GS2} = 2, V_{GS3} = 3.5, V_{GS4} = 4.5$

Sinh viên tự chọn loại đối tượng để thay đổi các giá trị tham số trong bảng trên.

Đồ thị mẫu

Bài 3

(*Dành cho App Designer) Thiết kế ứng dụng máy tạo dạng sóng có giao diện như sau



Hướng dẫn:

- Sử dụng các hàm sin, square, sawtooth để tạo dạng sóng
- Tạo một hàm tính toán và vẽ lên axes

-Trong các Callback của đối tượng gọi đến hàm tính toán ở trên

Bài 10: THỰC HÀNH TỔNG HỢP

Yêu cầu:

Mỗi nhóm tối đa 6 thành viên. Thực hiện trên Matlab App Designer để giải quyết các bài toán phương pháp tính bao gồm giao diện và luận lý. Nội dung như sau:

Giao diện Tìm nghiệm

Giao diện Nội suy

Giao diện Hồi quy

Nghiệm	Nội suy	Hồi Quy	Đạo hàm	Tích phân	Giới thiệu Nhóm	+
Nhập dữ liệu x Nhập dữ liệu y Chọn phương pháp hồi quy: Tuyến tính, hàm mũ, Logarit Kết quả phương trình hồi quy Nhập giá trị cần dự đoán Vẽ đồ thị hàm hồi quy và dữ liệu thực Kết quả dự đoán						

Giao diện Đạo hàm

Nghiệm	Nội suy	Hồi Quy	Đạo hàm	Tích phân	Giới thiệu Nhóm	+
Nhập dữ liệu x Nhập dữ liệu y Hoặc Nhập hàm số Nhập bước h Chọn giá trị sai số $O(h)$ hoặc $O(h^2)$ Chọn phương pháp đạo hàm: Xấp xỉ tiền, lùi, trung tâm Nhập giá trị cần tính đạo hàm Kết quả						

Giao diện Tích phân

The screenshot shows the 'Tích phân' (Integration) tab. It features a top navigation bar with tabs: 'Thử nghiệm', 'Nội suy', 'Hồi Quy', 'Đạo hàm', 'Tích phân', 'Giới thiệu Nhóm', and a '+' icon. The main area contains the following elements:

- Input fields: 'Nhập dữ liệu x', 'Nhập dữ liệu y', 'Nhập hàm số', and 'Nhập N'.
- Buttons: 'Chọn phương pháp Tích phân: Hình thang, Simpson 1/3, Simpson 3/8', 'Nhập cận để tính tích phân', and 'Kết quả'.
- Text: 'Hoặc' (Or) is placed between the 'Nhập dữ liệu y' and 'Nhập hàm số' fields.

Giới thiệu nhóm

The screenshot shows the 'Giới thiệu Nhóm' (Group Introduction) tab. It features the same top navigation bar as the previous tab. The main area contains a single text box with the placeholder text: 'Giới thiệu thành viên, nhiệm vụ, giới thiệu nhóm'.

Lưu ý:

- Ứng dụng có nhiều Tab, mỗi tab là một chủ đề
- Giao diện mỗi Tab có thể thay đổi tùy theo mỗi nhóm, có thể thêm chức năng
- Các phương pháp được viết trên hàm riêng và tổng hợp với nhau